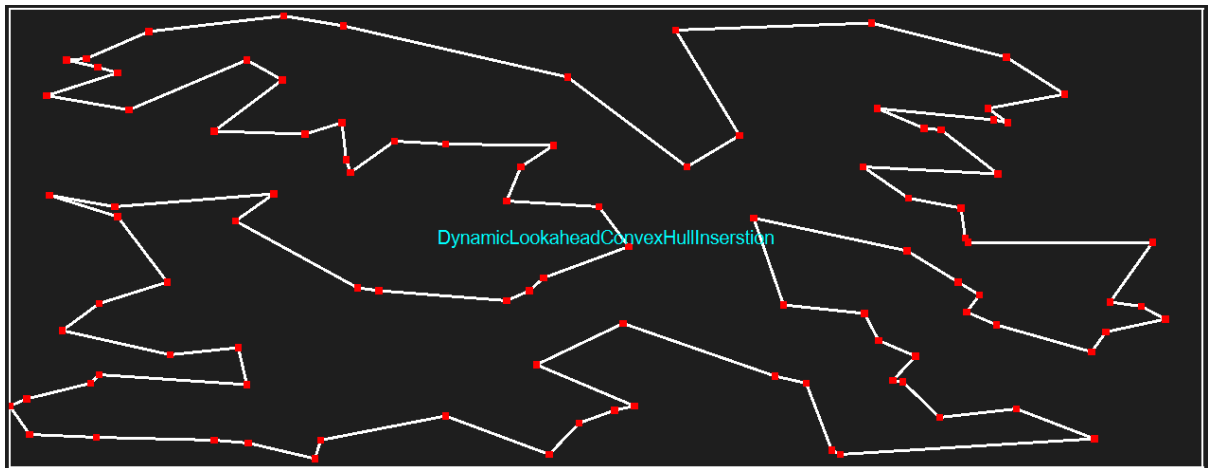

NEW LOOK-AHEAD HEURISTIC FOR THE TRAVELLING SALESMAN PROBLEM



Author: Andrew Foot, 2177545

Supervised By: Dr Rajesh Chitnis

Department of Computer Science
University of Birmingham

Word Count: 13,731

2025

Abstract

The Travelling Salesman Problem(TSP) is one of the most well-known optimisation problems in graph theory due to its straightforward formulation and wide range of real-world applications. However, as an NP-Hard problem, finding exact solutions is computationally infeasible for large instances, leading to the development of various heuristics that provide sub-optimal, but practical solutions.

This report examines and implements the enhancement of look-ahead mechanisms to the Cheapest Insertion Heuristic(CIH) to create two new algorithms, balancing computational efficiency with improved solution quality for the Symmetric Travelling Salesman Problem(STSP). To achieve this, in-depth research and computational testing with the aid of a substantial software engineering project are presented in this report.

When designing a new algorithm, the theory looks at the problem needing to be solved, new issues that get introduced, and its time complexity. At the same time, software engineering looks at implementing the design and making logical improvements to decrease the time complexity. Both are vital components covered in this report.

Evaluating any new algorithm requires a solid methodology and rigorous testing of itself and its performance against others in its class. Software has been designed and developed for empirical testing, visual inspection, and data analysis to impartially conclude the results of a new Look-Ahead Cheapest Insertion Heuristic.

Contents

1	Introduction	6
1.1	Motivation	8
1.2	Aims	8
2	Legal, Social, Ethical, and Professional Issues	8
3	Literature Review: Travelling Salesman Problem	9
3.1	Exact Solution	13
3.1.1	Concorde TSP Solver	14
3.2	Greedy Tour Construction Heuristic	17
3.2.1	Nearest Neighbour	18
3.2.2	Cheapest Insertion Heuristic	18
3.2.3	Convex Hull	19
3.3	Existing Look-Ahead Solution	20
3.3.1	Nearest Neighbour Look-Ahead	20
3.3.2	Dubins Look-Ahead TSP Solver	22
3.4	Tour Optimisation Heuristic	22
3.4.1	2-opt	23
3.5	Software Engineering Optimisation	23
3.5.1	Caching	23
3.5.2	Partitioning	24
3.5.3	Parallelisation	27
4	Research: Look-Ahead Heuristic	27
4.1	Problem To Solve	27
4.2	Static Look-Ahead Cheapest Insertion Heuristic(SLACIH)	28
4.2.1	Pseudocode	32
4.3	Pitfalls With New Solution	33
4.4	Dynamic Look-Ahead Cheapest Insertion Heuristic(DLACIH)	34
4.4.1	Static Dynamic	34
4.4.2	Adaptive Dynamic	36
5	Methodology	38
5.1	Empirical Testing	38
5.2	TSPLib	39

6	System Requirements	40
6.1	Functional Requirements	40
6.2	Non-Functional Requirements	42
6.3	C++	42
6.3.1	API & Interfaces	43
6.3.2	Template Metaprogramming	44
6.3.3	QT6	47
6.3.4	C++ Libraries	48
6.4	Hardware	48
6.4.1	Consistency	49
6.4.2	Multi-Threading	49
6.4.3	Memory	49
7	Design	49
7.1	TSP Library API	51
7.1.1	TSP File	52
7.2	TSP Data API	53
7.3	TSP Modules Interfaces	54
7.4	TSP Projects	55
7.4.1	Empirical Testing Suite	56
7.4.2	TSP GUI	59
7.4.3	Unit Test	60
7.4.4	TSP File System	61
8	Implementation	62
8.1	Polymorphism	62
8.2	Template Metaprogramming	64
8.3	API	66
8.4	Static Look-Ahead Cheapest Insertion Heuristic	67
8.5	Dynamic Look-Ahead Cheapest Insertion Heuristic	71
8.6	Gradient Descent	71
9	Project Management	72
10	Unit Testing	73
10.1	Basic Types	74
10.2	Quad Tree	74

10.3 Look-Ahead Algorithm	75
10.4 JSON Conversion	76
11 Evaluation	77
11.1 Functional & Non-Functional Requirements	77
11.2 Empirical Testing Results	80
11.3 TSPLib Results	83
11.4 Time Analysis	85
12 Conclusions	88
12.1 Future Work	89
12.2 Closing Remark	90

1 Introduction

The Travelling Salesman Problem(TSP) is one of mathematics' most studied optimisation problems due to its simple problem definition and wide use case^[1]. The goal is to find the shortest route that visits every node exactly once. It is used in many fields and industries, as its graphing properties can be applied to various problems, leading to significant interest from both theorists and practitioners.

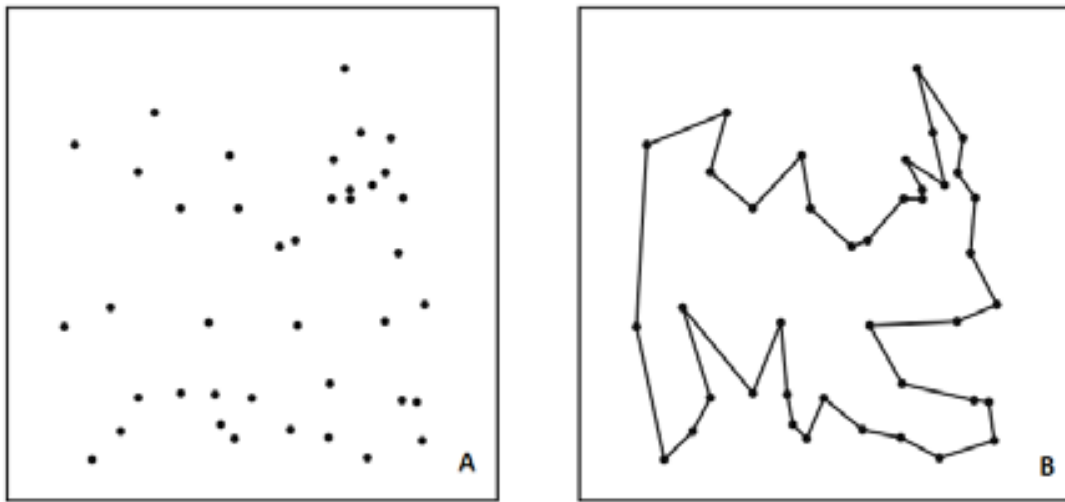


Figure 1: TSP And Solution
[2]

However, due to the NP-Hardness property of the problem^[3], an exact solution can be computationally infeasible. Therefore, approximations are a suitable replacement for some applications where an exact solution is not required. These algorithms are called heuristics due to their approximating solutions and quick results.

Look-ahead(LA) algorithms are an algorithm design type with no formal definition. They are not a new concept, as they are primarily found in decision problems, but range from backtracking to forward checking to combinatorial search.

This report defines look-ahead as a sub-procedure that recursively checks how a greedy prediction affects future options and applies a semi-intelligent outcome with foresight. Applying foresight to avoid the common pitfalls of greedy algorithms introduces its own challenges because a look-ahead heuristic is still classified as a greedy approach; under the wrong circumstances, it will produce suboptimal results.

Dynamic look-ahead is an extension to the look-ahead algorithm design that will be explored

to help resolve and improve the issues facing a standard approach to look-ahead. It aims to achieve this by changing the look-ahead search depth dynamically at runtime based on the problem at large through various metrics.

This project will apply these algorithmic designs to the existing Cheapest Insertion Heuristic(CIH) greedy algorithm to create two new algorithms, Static Look-Ahead Cheapest Insertion Heuristic and Dynamic Look-Ahead Cheapest Insertion Heuristic. Its unique design, implementation, and performance are compared to other classical algorithms and then evaluated.

An in-depth review of the Travelling Salesman Problem, existing solutions, and software optimisation techniques is conducted for the project to consider. This report researches the logical steps required to apply a look-ahead design to CIH and the potential benefits and issues that come with a look-ahead design. A scientific methodology is devised for an organised approach to the testing, evaluation, and conclusion of the findings of this project.

A significant software development project is included to verify and empirically test theoretical ideas. State-of-the-art methods and techniques are employed to ensure unbiased results and optimal performance. Algorithmic designs are coded with accuracy and are deterministic for reliability and scrutiny of the results. The same core library code is used to execute multiple runtime solutions to fit the various needs of the project.

When evaluating a new algorithm, multiple methodologies are explored. Empirical testing on new randomly generated problems is conducted against existing conventional greedy heuristics, and then data analysis is applied. A famous testing library, TSPLib^[4], of optimally solved problems serves as a ground truth to measure how the new designs perform against exact solutions.

Graphs and renders of solutions have been developed for verification and visual evidence. Allowing for easy comparison between solutions and quick data analysis is vital for later evaluations. Data collection and processing for statistics and scientific research are embedded in the analysis of each solution.

White box unit testing covered the codebase from bugs and unexpected behaviour changes. Waterfall project management helped ensure the project stayed on track and reduced the amount of refactoring and backtracking.

1.1 Motivation

This report will research, design, test, and evaluate the use of a look-ahead design on an existing greedy heuristic to create an improved solution over that of its greedy heuristic counterpart. This report will not attempt to solve the Travelling Salesman Problem or discuss the possibility of the P versus NP problem.

1.2 Aims

This report aims to tackle the challenges present when creating a new algorithm and to follow current research in methodology to produce a worthwhile and fair conclusion. Following this, some aims have been outlined, and each will be independently evaluated at the end of this report in Section 12.

1. Design a new look-ahead algorithm that runs in polynomial time based on the greedy Cheapest Insertion Heuristic
2. Fairly and objectively test and evaluate each performance metric of a heuristic algorithm
3. Develop and maintain a working code base to run various algorithms reliably through a standard API

2 Legal, Social, Ethical, and Professional Issues

The British Computer Society (BCS) Code of Conduct^[5] outlines the legal, social, ethical, and professional issues in the fields of computer science and information technology. This project complies with all the conducts stated by BCS and does not produce any issues.

- Public Interest:
 - The research is only intended to benefit the public
- Professional Competence and Integrity:
 - This work is solely my own, and all third-party libraries used comply with their respective licensing agreements
 - Due Diligence has been conducted into the current field research
- Duty to Relevant Authority:

- The data and information shared in this report are faithful for all relevant authorities to read
- Duty to the Profession:
 - Throughout the project, communication with my supervisor, Dr Rajesh Chitnis, has been professional and respectful

3 Literature Review: Travelling Salesman Problem

The Travelling Salesman Problem(TSP) is two problems in graph theory: a decision and an optimisation problem^[6]. However, it is most famous for its optimisation version, which is what this report covers. The most basic version of the optimisation problem is finding the shortest Hamiltonian cycle from a set of nodes(vertices) with non-negative weighted edges(links) to every other node(complete graph)^[1]. This can be formalised in graph theory with ordered pairs $G = (V, E)$ ^[7].

- G , a graph
- V , a set of nodes
- $E \subseteq \{\{x, y, w\} \mid x, y \in V, w \in \mathbb{R}_{\geq 0}, x \neq y\}$, a list of weighted edges from node x to node y

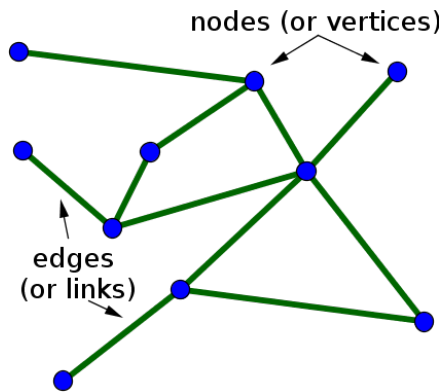


Figure 2: Graph Theory Diagram
[8]

A Hamiltonian cycle is defined as from a graph, $G = (V, E)$, an ordered list of edges creating a continuous path where every node is visited exactly once^[9]. In the research for TSP, the route can be defined as a 2-dimensional adjacency matrix, A_{ij} , of boolean values to represent if an edge is present in the cycle^[10]; It can also be listed as an ordered list^[11], P_i , where $P \subseteq E$. The

cost function of the cycle and the Travelling Salesman Problem is the summation of all the weights of the edges in the list P and is what is trying to be minimised^[1]:

$$\min(\sum_{i \in P} \text{weight}(P_i)) \quad (1)$$

Hamiltonian cycle has some formal constraints in graph theory^[9]:

- $|V| \geq 3$, There must be at least three nodes to create a cycle
- $|P| = |V|$, The list must have as many edges as there are nodes in the graph
- $\forall P_i : y_i = x_{i+1}$, Each edge in the list must create a continuous path
- $\forall P_i \forall P_j : i \neq j, x_i \neq x_j, y_i \neq y_j$, Each node can only have two edges connecting to it in the route

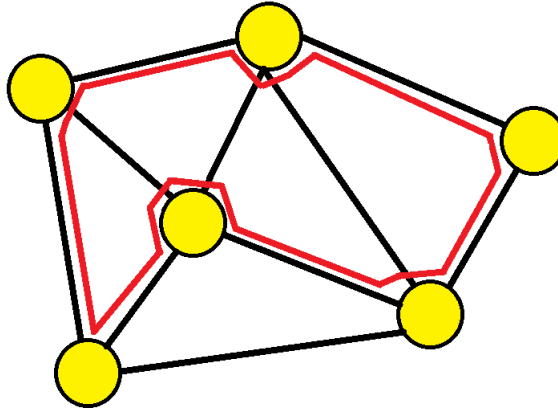


Figure 3: Hamiltonian Cycle
[12]

While the adjacency matrix is more common in research, an ordered list is more memory efficient when programmatically storing a route. The list can store a list of edges or the order in which the nodes are visited. The latter is more popular as it is more memory efficient, but has slower read times. This is possible because the graph must be fully connected, therefore, the taken edges can be deduced from the adjacency matrix(Figure4).

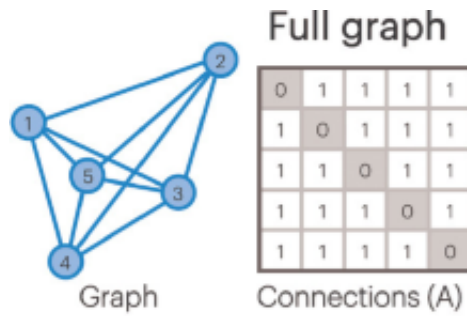


Figure 4: Adjacency Matrix
[13]

Over the years, constraints and variants of the classic optimisation problem have been conjured to fit the specific needs of researchers. Most are derived from changes to the type of graph the problem is solving^[14]: symmetric, asymmetric, Euclidean, Manhattan, two dimensions, three dimensions, and many more. But there are also new constraints like Dubins TSP^[15], which specifies that the route has a minimum turning radius, or K-TSP^[16], which introduces a starting hub and k number of salesman/routes, or TSP with Time Windows(TSPTW)^[17], which adds temporal constraints to the problem.

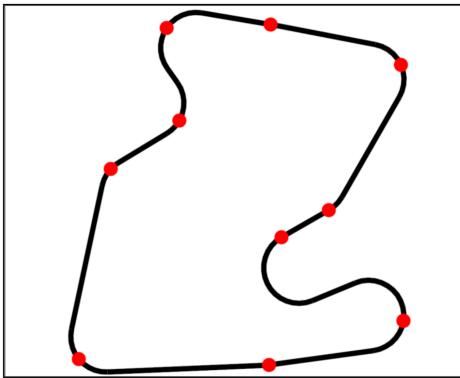


Figure 5: Dubins TSP
[18]

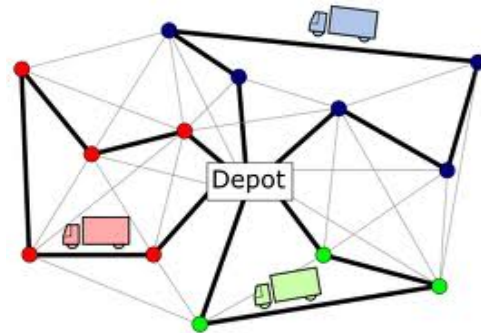


Figure 6: K-TSP
[19]

A popular version is the symmetric two-dimensional Euclidean Hamiltonian Travelling Salesman Problem, which is used for the project for its adaptability to other problems. This is the most basic version and can be a catalyst for many other problems. It can be broken down into:

- Edges between two nodes that have the same weight bidirectionally
- Nodes are positioned on a two-dimensional plane

- The distance between nodes is calculated using a straight distance
- The route is made up of only one Hamiltonian cycle

The TSP optimisation problem belongs to the NP-Hard classification of problems^[3], as no algorithm exists for creating or verifying a solution in polynomial time. This is verifiable because there is a polynomial time reduction from the Hamiltonian cycle problem to the Travelling Salesman Problem^[20]. All variants of TSP are also NP-Hard, as a polynomial-time reduction exists to handle the additional constraints^[21].

When developing routes from nothing, two sub-procedures are used to generate a TSP solution. The different procedure determines if the algorithm is a tour constructor, a tour optimiser algorithm, or both^[22]. Tour constructor algorithms focus on adding unvisited nodes to a route in an optimal manner^[23]. Tour optimiser algorithm focuses on improving an existing route by alternating the edges being taken^[24]. The two sub-procedures are:

- Construction: Iteratively adds an unvisited node to the route.
- Pairwise Exchange^[25]: Removing two edges from a route and reconnecting the resulting segments in an alternate way that still forms a cycle.

Table 1: Sub-Procedures

When geometric graphs are used, the Hamiltonian cycle has a property that the optimal solution will never have two edges that intersect each other due to the triangle inequality^[26]. In Euclidean geometry, the triangle inequality states that the length of any two sides of a triangle is always greater than or equal to the length of the remaining side. Therefore, when a route is treated as a polygonal chain, a closed polygonal chain that does not form any triangles is strictly better than a polygonal chain that self-intersects and forms a triangle^[27]; this is the basic principle 2-opt is built upon.

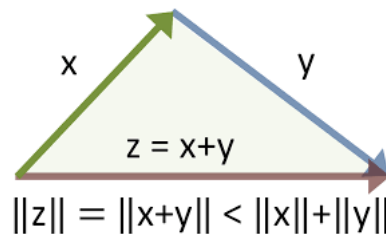


Figure 7: Triangular Inequality
[28]

3.1 Exact Solution

An exact or optimal solution is the route with the global minimum cost for a given graph; no other valid route has a lower cost^[1]. A graph can have multiple exact solutions if two or more routes have equal costs^[29]. An exact solution to the TSP is ideal in most situations, but it is sometimes a requirement in areas of research, such as DNA sequencing^[30].

For a graph with n nodes, there are $O(n!)$ possible routes^[31]. A simple approach to finding the optimal route would be a brute-force search, which calculates every possible combination of routes and stores the best result. This was taught in the year three Algorithms and Complexity module. However, with $O(n!)$ possible routes and $O(n)$ time to calculate the route cost, this becomes computationally infeasible after $n \geq \approx 20$ ^[32], due to exponential time growth^[33].

A similar mathematical problem is the sequencing problem^[34], which has an arbitrary cost function and a recurring scheme; this matches the TSP brute-force search. A dynamic programming solution exists for the sequencing problem^[35] and can be applied to brute-force search^[36], which can cut down on the number of routes that need checking to $O((n-1)2^{n-2})$ ^[36]. This is a significant improvement, but still means that problems after $n \geq \approx 57$ are infeasible.

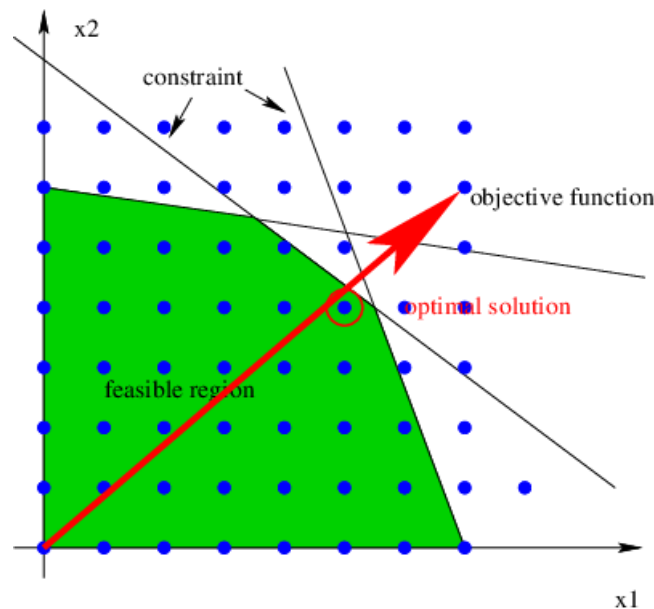


Figure 8: ILP Problem
[37]

Another method, not covered in the Algorithms and Complexity module, for solving exact solutions is solving TSP as an Integer Linear Programming (ILP) problem^[38]. ILP is an NP-hard

optimisation problem with an objective function and a set of linear equations that must be satisfied. Some of these linear equations are developed from the definitions and constraints stated at the start of Section 3. The worst-case time complexity for ILP is $O(n!)$ ^[39], but it can be significantly lower, depending on the problem^[40] (covered in Section 3.1.1).

3.1.1 Concorde TSP Solver

Due to the need for exact solutions to the Travelling Salesman Problem, research into optimising the development of exact solutions has been in development since the 1970s. William Cook, a researcher from the University of Waterloo, has made significant strides in this area over the past two decades with his team's Concorde TSP Solver^[41], which is widely regarded as the best solver available^[40]. Concorde holds the record for the largest TSP problem solved optimally at 85,900 nodes^[42] and is the first solver to obtain an exact solution to every problem in the TSPLib collection^[43]^[44], along with winning many awards for solving open problems^[45]. It uses all of the most recent research and techniques, which this project researched and will explain.

Concorde is an extension of the University of Waterloo's research into solving Integer Linear Programming with their QSOPT Linear Programming Solver^[46]. Concorde solves problems by taking an existing good heuristic solution and relaxing the linear constraints to make the linear problem easier to solve^[47]. It can relax constraints using the Cutting-Plane Method, Branch-and-Bound, and Branch-and-Cut algorithms^[48].

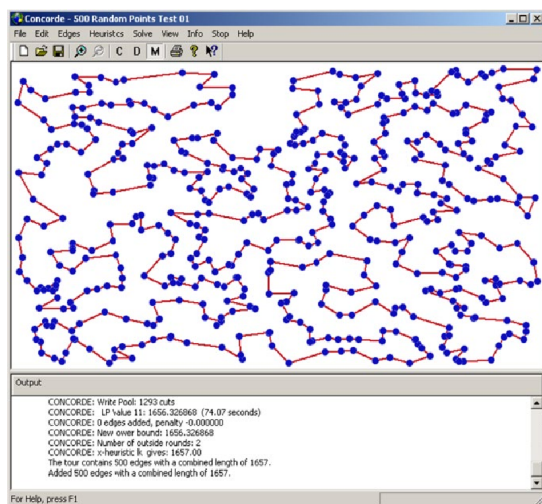


Figure 9: Concorde TSP Solver
[49]

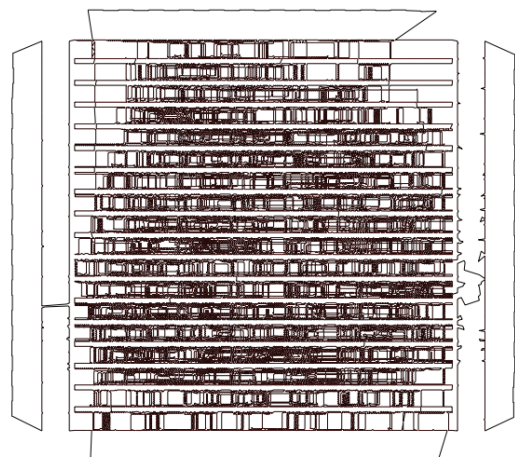


Figure 10: 85900 Node Solution
[50]

Branch-and-Bound(B&B)(Figure11) is an optimisation algorithm that solves Integer Linear Programming(ILP) problems by systematically branching the search space into smaller sub-problems and bounding non-promising branches^[51]. It achieves this by recursively dividing the solution space and using higher and lower bounds to eliminate sub-problems that cannot yield a better integer solution - a process known as “pruning”^[52]. The best-known integer solution typically sets the upper bound, often obtained using TSP heuristic algorithms. In contrast, the lower bound is derived from solving relaxed Linear Programming(LP), where integer constraints are temporarily ignored^[46]. This relaxed LP provides a bound that helps determine whether a branch should be explored further or pruned.

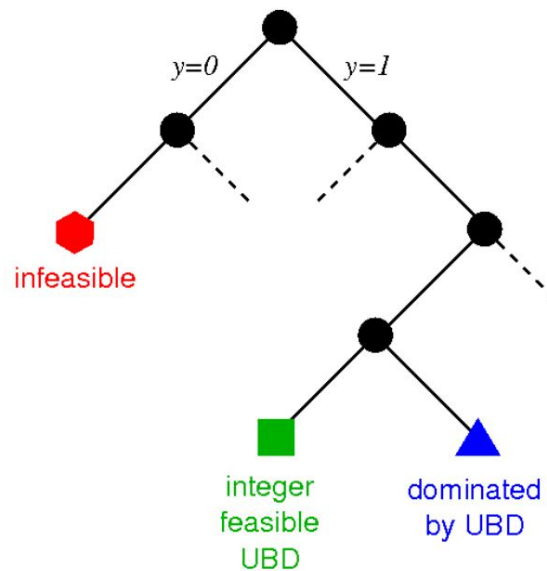


Figure 11: Branch And Bound
[53]

The Cutting-Plane Method(Figure12) is a technique for solving a Mixed-Integer Linear Program(MILP) by iteratively solving its Linear Programming(LP) relaxation and adding constraints(cuts) until an integer solution is found^[54]. First, it relaxes the problem to non-integer constraints and is solved as Linear Programming^[55]. If the solution contains non-integer values, new hyperplanes(cuts) are added iteratively to the constraints until an integer solution is found^[44]. The benefit of this approach is that LP is more straightforward to solve than MILP, but finding substantial cuts iteratively can be computationally expensive^[44].

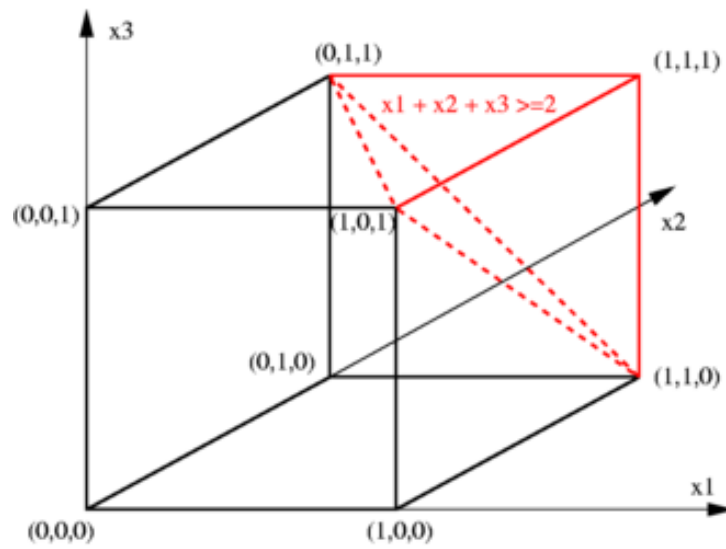


Figure 12: Cutting-Plane Method
[56]

Branch-and-Cut combines (Figure 13) Branch-and-Bound and Cutting-Plane methods into one^[57]. It relaxes the problem to linear programming constraints, and in the pruning of Branch-and-Bound, it adds cuts back into the constraints. Added cuts can be global (cuts valid across the entire problem) or local (specific to a sub-problem in the search tree), and multiple branching strategies can be applied due to the relaxation^[58]. The benefit of Branch-and-Cut is its scalability for large-scale problems ($n \geq \approx 1000$)^[59], making it the go-to approach for solving ILP.

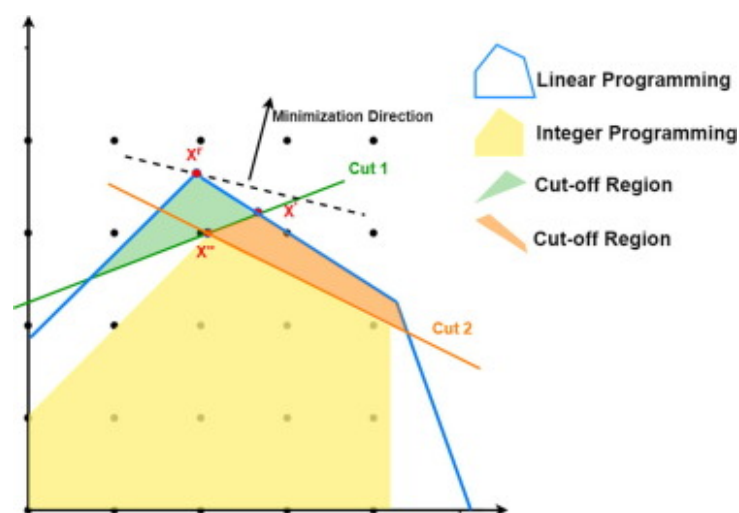


Figure 13: Branch And Cut
[60]

For Concorde TSP Solver, the efficiency is directly related to the starting solution provided, as it acts as the upper bound for Branch-and-Bound^[44]. A tighter upper bound will reduce the number of valid branches by pruning more sub-problems. Therefore, research is still being applied to finding high-quality heuristics for the knock-on effect of finding a quicker exact solution in the Concorde TSP Solver.

Concorde's time complexity is not fixed because it depends on several factors, such as the starting heuristic solution and the specific instance of the problem^[47]. The worst-case complexity for branch-and-cut is exponential, $O(2^n)$ ^[61], but practical performance is often significantly better, with standard hardware able to achieve $n \geq \approx 1000$ ^[48]. However, many software engineering optimisations are applied to hasten the solution time, allowing massive problems to be solved on supercomputers.

3.2 Greedy Tour Construction Heuristic

Sometimes, an approximate solution to the TSP is adequate for users when time complexity or computational resource constraints are a concern^[62]. Amazon delivers approximately 1.6 million packages daily, with the average driver making 180 stops on their shift. This poses a logistical problem when a package could be out for delivery within a shorter time frame than the time to produce an exact solution. Another case is that many real-time path-finding applications must decide within seconds and are often limited by embedded hardware. Therefore, reliance on approximations for quick but quality solutions is still desirable for specific applications.

Heuristics are problem-solving strategies that use educated guesses to find approximate solutions quickly^[63]. Heuristics are algorithms that follow structured logical steps; however, these logical steps take shortcuts, often trading completeness for speed. Greedy algorithms are a type of heuristic that makes local optimal choices to approximate global optimal solutions^[64]; they are susceptible to pitfalls that cannot be solved later^[65].

For the Travelling Salesman Problem, developing a route from a problem requires a construction strategy; these strategies are called tour constructions^[22]. A tour can be constructed in many ways and produce different results based on the starting location and variables. It usually starts with an empty route and randomly selects a node as the starting location; however, more innovative beginnings are available^[22]. Most tour constructors will iteratively add one node that is not already in the route to the route at the 'head' or anywhere along the route.

Greedy tour constructors typically have polynomial time complexity and a fixed time complexity

due to their exhaustive searching characteristics. Further improvements, such as caching and partitioning, can expedite the runtime, but the time complexity will remain the same. There are no guarantees on the lower bound of the solution cost against the optimal solution^[66], as we would need to solve the P versus NP problem^[66], which remains unproven.

3.2.1 Nearest Neighbour

The Nearest Neighbour Search(NNS) greedy algorithm is one of the oldest and most straightforward approaches for approximating the TSP. It was taught in the Algorithms and Complexity module. The origin of the heuristic is uncertain, but due to its simple nature, its approach is the most logical when thinking of a new method. The steps for the algorithm are as follows^{[67] [23]}:

1. Select a random node as head and add it to the list of visited nodes
2. Check every unvisited node:
 - 2.1. select the unvisited node that has the lowest cost to the head
3. Make the selected node the new head and add it to the list of visited nodes
4. Repeat step 2 until there are no unvisited nodes
5. Close the cycle by adding an edge from the current head to the starting node

The time complexity for nearest neighbour is $O(n^2)$ ^[22]. It does $n - 1$ iterations from step 2 to step 4, and each iteration checks $O(n)$ nodes as a possible shortest addition.

3.2.2 Cheapest Insertion Heuristic

The Cheapest(sometimes called shortest) Insertion Heuristic(CIH) takes a slightly more logical approach to constructing tours and was researched for this project. The addition of nodes to the route, can be added anywhere into the route^[23]. The insertion cost formula for CIH calculates the breaking of the current route edge and the addition of two new edges. As nodes are added anywhere into the route, the starting route does not just have to be a single node; more advanced setups, such as Convex Hull^[22] or Shrinking Blob^[68], can be adopted to improve the result^[69].

The insertion cost formula, where $d(a, b)$ is the cost of the edge between node a and b , x is the node to be added and a, b are consecutive nodes already in the route^[23]:

$$\text{Cost}(x, a, b) = d(x, a) + d(x, b) - d(a, b)^{[70]} \quad (2)$$

The steps for the algorithms are as follows^{[70] [23]}:

1. Generate a starting route(random node or convex hull)
2. For each unvisited node,
 - 2.1. For every position in the route
 - 2.1.1. Calculate the cost function of adding the node to the route position
3. Add the selected node in the position with the lowest cost function
4. Repeat step 2 until there are no unvisited nodes

The time complexity for the cheapest insertion is $O(n^3)$ ^[22]. Its run-time changes based on the size of the starting route, but at worst, $n - 1$ iterations are run from steps 2 to 4. Each iteration is $O(n^2)$, as $O(n)$ unvisited nodes must calculate the cost function against $O(n)$ positions in the route. Also, calculating the cost function(Equation2) is three times more expensive than the Nearest Neighbour cost function, making CIH a significantly slower algorithm.

3.2.3 Convex Hull

Convex Hull is a tour construction algorithm that does not guarantee a complete solution; instead, it generates part of a route^[71]. The idea of a Convex Hull is to encapsulate all the points of a problem within a shape known as a convex hull^[72]. Convex hulls utilise geometric locational properties of nodes, so only n-dimensional TSP problems based on Euclidean or Manhattan distance are suitable for using Convex Hulls^[71].

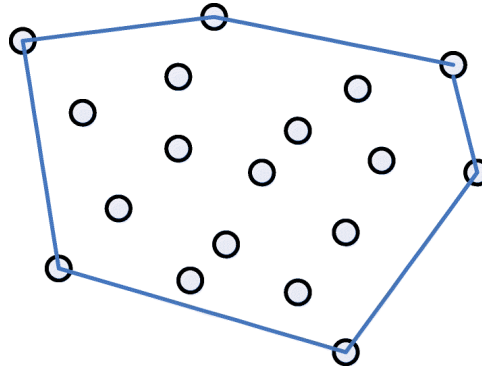


Figure 14: Convex Hull
^[73]

There are many algorithms for generating the structure, like Gift wrapping^[74], Graham scan^[75], and Chan's Algorithm^[76]. Gift wrapping is the simplest to understand but the least efficient; the steps are^[74]:

1. Find the northernmost point, add it to the route, and make it the head
2. For every node not in the route calculate the gradient to the head
3. Add the node that forms the smallest counter-clockwise turn to the route and make it the new head
4. Repeat step 2 until the head is back to the starting node

The number of iterations is not fixed per problem, so h defines how many nodes are in the convex hull; h can be any size between $n \geq h \geq 3$. Each time h increases by one, every node needs to be compared. Therefore, the gift-wrapping algorithm has a worse time complexity of $O(n^2)$, but there exist more complex algorithms that run in $O(n \log(n))$ ^[75].

The benefit of doing a Convex Hull is that it is a strong starting point for other heuristics^[71]. The order of the nodes in a Convex Hull is guaranteed to be in the order of the optimal solution, as any other order of the partial route would have to cross over itself. This improves the results produced and quickens other tour constructors by reducing the number of nodes that need to be added. Algorithms that use Convex Hull address it in the name by placing it in the front, e.g. Convex Hull Cheapest Insertion Heuristic(CHCIH). This is because algorithms like CHCIH perform significantly better than CIH just by changing the starting route.

3.3 Existing Look-Ahead Solution

Look-ahead algorithms are valuable for dealing with decision problems^[77]; they are prevalent in tree searching and constraint solving^[78]. The premise is to evaluate future outcomes before making a decision by analysing future states to guide decision-making^[79]. Look-ahead algorithms are not known for their speed, as additional computation power is required to evaluate future states, but they make up for it with above-average results^[80]. Research for look-ahead in TSP focuses primarily on adapting decision problems to different versions of TSP that can leverage this research^[81].

3.3.1 Nearest Neighbour Look-Ahead

A paper by the Indian Institute of Technology called "Improving Greedy Algorithms by Lookahead-Search"^[80] researches the use of look-ahead for solving the 0/1 Knapsack problem^[82].

It is an optimisation decision problem for finding the optimal subset of items that maximises a cost function within a given constraint. The researchers devised that the decision problem can be formulated as a tree that can be recursively traversed to a look-ahead depth of k with existing greedy algorithms^[80]. A tree structure is used because if it reaches a leaf node, then a constraint has been reached, and the algorithm should return. The researchers included pseudocode that worked for any generic greedy algorithm where the problem could be displayed as a tree^[80].

Algorithm 1 lookahead-search(k : integer, T : tree, G : greedy algorithm)

```

1:  $n$  : node    ▷  $k$ -depth lookahead search in  $T$  using  $G$  as the underlying greedy algorithm.
2:  $path := \emptyset$ 
3:  $n := \text{root}(T)$ 
4: lookahead( $k, n, G$ )
5: print  $path$  as the solution

```

[80]

Where Procedure *lookahead* is defined recursively as follows:

Algorithm 2 lookahead(k : integer, n : node, Greedy : greedy algorithm)

```

1:  $l, m$  : node
2: if  $n$  is a leaf node then return
3: else
4:   Get  $m \in \text{des}(n, k)$  such that
5:    $\text{cost}(n \rightarrow m) + \text{Greedy}(m) = \min_{l \in \text{des}_{n,k}} \{\text{cost}(n \rightarrow l) + \text{Greedy}(l)\}$ 
6:    $m := \text{succ}(n \rightarrow m)$ 
7:    $path := path + \langle n, m \rangle$                                 ▷ augment path; path is global
8:   lookahead( $k, m, \text{Greedy}$ )

```

[80]

The researchers applied this pseudocode to the nearest neighbour greedy algorithm because it can be easily applied to a tree structure. The head node is the root state, and the greedy algorithm can be applied recursively to generate the states of the children nodes.

The researchers found that only minor improvements were made when applying the look-ahead design, however, it can produce better results than its greedy algorithm counterpart^[80].

3.3.2 Dubins Look-Ahead TSP Solver

A paper by Automation Science and Engineering, titled “Discretization-Based and Look-Ahead Algorithms for the Dubins Traveling Salesperson Problem”^[83], investigates the use of look-ahead for solving the Dubins version of the TSP. Dubins Travelling Salesman Problem(DTSP) adds the constraint that the route has a minimum turning radius^[15]; this is to mimic the constraint that something like a fixed-wing aircraft faces.

The researchers developed a look-ahead algorithm for DTSP, where L is the look-ahead depth^[83]:

1. Get a solution for a corresponding Euclidean TSP
2. Selected a random node as the head, h , and generate an initial heading to $h + 1$
3. Iterate h for all nodes in the solution
 - 3.1. Brute force calculates the shortest Dubins route for the subset, $h - 1$ to $h + L - 2$ nodes, where the heading of $h - 1$ is fixed
 - 3.2. Permanently set the heading of h
4. Return the newly organised solution

The look-ahead part of this algorithm involves brute-forcing a small subset of the route and setting only the heading of node h . $h - 1$ heading has to be fixed to allow for the continuous build-up of a Dubins route. Brute forcing a subset of the problem does not result in an exact solution because faults are introduced in the combination step, but it does provide a better result than if a greedy approach were taken.

3.4 Tour Optimisation Heuristic

Sometimes, the initially constructed route is not good enough, and simple improvements can be made to the route’s ordering to achieve a better result^[22]. This is where Tour Optimisation algorithms are used to improve the ordering and refine the edges taken from an already complete cycle while maintaining the properties of the TSP^[22].

Tour Optimisation heuristics effectively resolve local pitfalls that have been taken, but they struggle to find global pitfalls. Optimisers rarely find the exact solutions to a problem, but they are valuable as they often find improvements on existing solutions cheaply. This report researches its design.

3.4.1 2-opt

The simplest optimisation heuristic is Croes's 2-opt local search algorithm in 1958^[10]. It utilises the property that any two edges crossing over each other are suboptimal and calculates whether swapping edges can produce a better result. For Symmetric graphs, the cost function below calculates if swapping two edges will benefit the route:

$$Cost(r_i, r_j) = d(r_i, r_j) + d(r_{i+1}, r_{j+1}) - d(r_i, r_{i+1}) - d(r_j, r_{j+1})^{[25]} \quad (3)$$

The steps for 2-opt are as follows^[10]:

1. For each pair of edges (i, j) , where i ranges from 0 to $n - 1$ and j ranges from $i + 2$ to n :
 - Calculate the cost of swapping the edges.
 - If the swap results in a negative cost, perform the swap.
2. Repeat step 1 until no swaps result in a negative cost.

The time complexity of 2-opt is often difficult to predict in real-world examples, as it is unknown how many iterations it will perform. Each iteration takes $O(n^2)$ ^[25], as it checks two nested For loops of combinations. Although 2-opt is likely to converge after only a few iterations, the worst-case scenario is $O(n^2)$ iterations due to the numerous combinations of swaps available for an edge before it reaches a location it has been in before. Therefore, the worst case for 2-opt is $O(n^4)$ ^[25], however if applying a limit of m iterations results in $O(m \cdot n^2)$.

3.5 Software Engineering Optimisation

When solving the TSP, the fundamentals of each algorithm remain the same, but data structures and algorithms can be applied to achieve a constant-factor speed-up. Each software engineering optimisation benefits the computational runtime for the average case, but the Big O time complexity remains the same in the worst case^[48].

3.5.1 Caching

Caching is the process of storing computational results to be used later for quick callback. When the problem is assigned as a list of nodes, the edge weights are not necessarily known at run time^[48]. Calculating edge weights can be expensive depending on the TSP problem

type(3D Euclidean, Global Coordinates, A* Pathfinding)^[84]. Therefore, a 2D Adjacency Matrix can store n^2 edge results. Two fill strategies can be applied to deal with caching:

- Full: Calculate every edge weight before trying to solve the problem
- Partial: See if the edge weight has been cached before; if not, calculate the edge

If it is known that an algorithm will scan most edges, doing full caching is ideal, as there is no overhead for checking if a cache miss has occurred. If an algorithm does not scan all the edges, partial caching helps reduce unnecessarily expensive calculations.

3.5.2 Partitioning

Partitioning is the process of bucketing nodes into groups based on the geometric location of the point^[48]. There are two use cases for partitioning in the Travelling Salesman Problem^{[85] [86] [87] [88]}:

- Closeness proximity: Reduce the search size of nodes based on proximity
- Creating smaller problems:
 1. Group sets of nodes together
 2. Solve each set
 3. Merge the solutions to solve the original problem

Closeness proximity is helpful when evaluating potential nodes in heuristics^[87]; it can cheaply exclude nodes as valid options. The partitioning algorithm's search time is typically $O(\log(n))$; therefore, reducing the search space usually results in an expedited solution without affecting the result.

Splitting a large problem into smaller problems can be useful as time complexity grows exponentially^[85] with problem size. By dividing the problem, each solution becomes more manageable, but the solution quality drops as decision boundaries are formed between the smaller problems, resulting in disjointed routing^[88]. It is merging of solutions step that introduce imperfections in the solution quality^[89].

There are many partitioning algorithms, but the most common are K-means Clustering^[90], Grid-based^[91], and Quad-Trees^[91].

- K-means Clustering: Given k clusters; distribute each cluster such that the nodes are evenly grouped between the clusters

- Grid-based partitioning: Split the graphing area into evenly distributed quadrilaterals and bucket the nodes to them
- Quad-Trees: Subdivides the graphing area into smaller and smaller quadrilaterals in a quad-tree structure; nodes are assigned at each layer of the tree

K-means clustering is expensive to calculate and progressively more complex as the number of clusters increases^[92]. However, many clusters are beneficial when partitioning the TSP problem, and K-means struggles when there is an even distribution of points^[90].

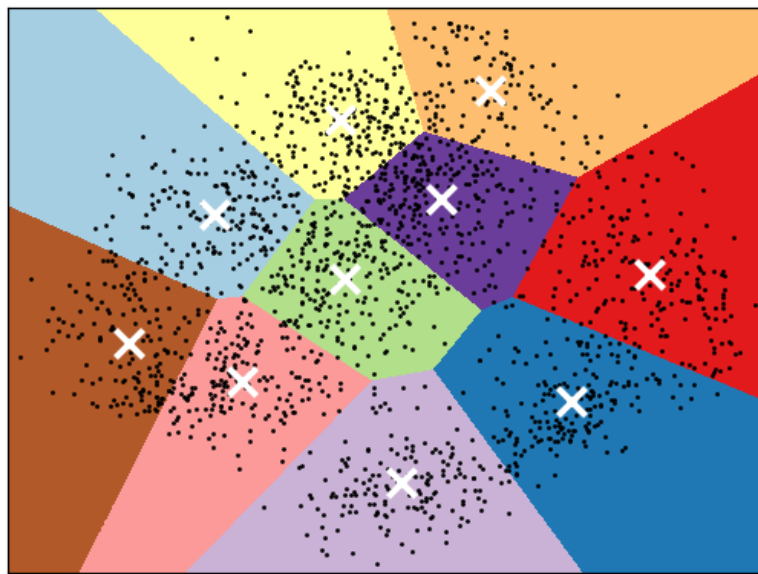


Figure 15: K-Means Partitioning
[93]

Grid-based partitioning is the most straightforward approach with the quickest search and creation times and works well for evenly distributed points. When the points are concentrated, grid-based partitioning fails to separate the points by grouping a large number of points in the same bucket.

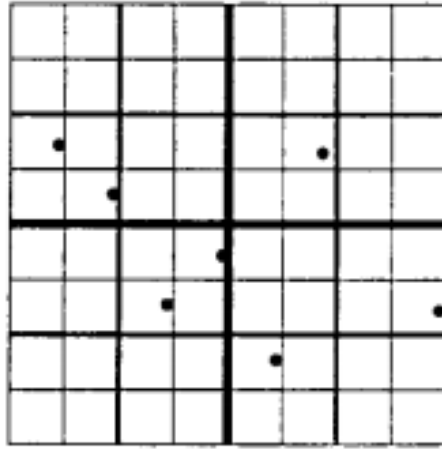


Figure 16: Grid-based Partitioning
[91]

Quad-tree partitioning takes a hierarchical approach by subdividing the problem based upon the points^[94]. As each point is assigned its own node in the tree, testing for closeness can be more precise, and the results are better for densely packed nodes^[91]. However, the construction is expensive^[94], there is an overhead in search time^[94], and it requires a significant amount of memory.

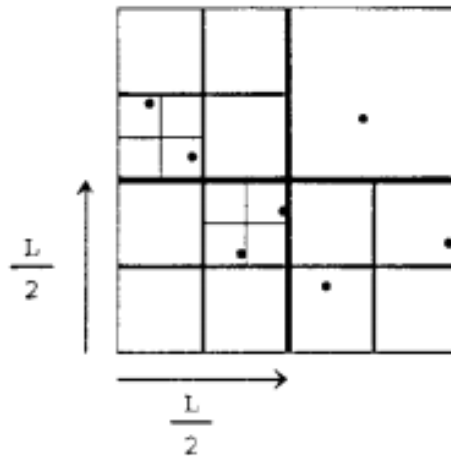


Figure 17: Quad-Tree Partitioning
[91]

Although partitioning has many benefits, it requires the data to have geometric properties. The benefits are also not guaranteed, especially when the nodes are densely packed.

Partitioning	Creation Time	Read Time	Partitioning Result
K-Means	$O(2nki)$	$O(1)$	Average
Grid-Based	$O(n)$	$O(1)$	Worst
Quad-Tree	$O(n\log(n))$	$O(\log(n))$	Best

3.5.3 Parallelisation

Parallel computing involves splitting calculations across multiple processors and executing them simultaneously^[95]. There are many challenges, mainly related to data handling and synchronisation^[96]. But the Travelling Salesman Problem lends itself well to parallelisation due to the iterative nature of finding the best fit and only changing the problem state once all options have been considered^[97]. Parallelisation is how cutting-edge technology like Concorde TSP Solver can solve massive problems using supercomputers^[44].

GPUs can also be applied to TSP algorithms^[44] due to their capabilities in matrix arithmetic and vectorisation programming. Two 2D Adjacency Matrices are multiplied together, and the grand sum is taken to calculate the length of the route, allowing for quick calculations. Researchers tested the speed difference for a few algorithms and recorded a dramatic reduction in run-time of 24.2 times quicker than their CPU counterparts^[98].

4 Research: Look-Ahead Heuristic

Look-ahead(LA) has many different definitions^[99], but for a deterministic system, knowledge of future outcomes(states) can be determined if the current state is known; this is defined as foresight, and it can be used to evaluate options more intelligently^[100].

As mentioned in Section3.3, look-ahead algorithms and their design have already been adopted to solve the Travelling Salesman Problem and its variants. In Section3.3.1, the greedy Nearest Neighbour Search algorithm was employed for the STSP to conform to the look-ahead algorithm proposed in their research. This is not possible for the Cheapest Insertion Heuristic.

4.1 Problem To Solve

Greedy heuristics rarely produce global optimal solutions because they only evaluate locally optimal decisions, whereas the exact solution makes trade-offs at the local level to achieve better gains globally^[65]. These pitfalls are often hard to spot and are practically irreversible once made, especially in TSP^[101].

The cause of local optimal pitfalls varies for each algorithm, but for the CIH(Section3.2.2), the

pitfall occurs when a cluster is chosen because a single node is closest to the route, whereas a better cluster construction is possible in another way.

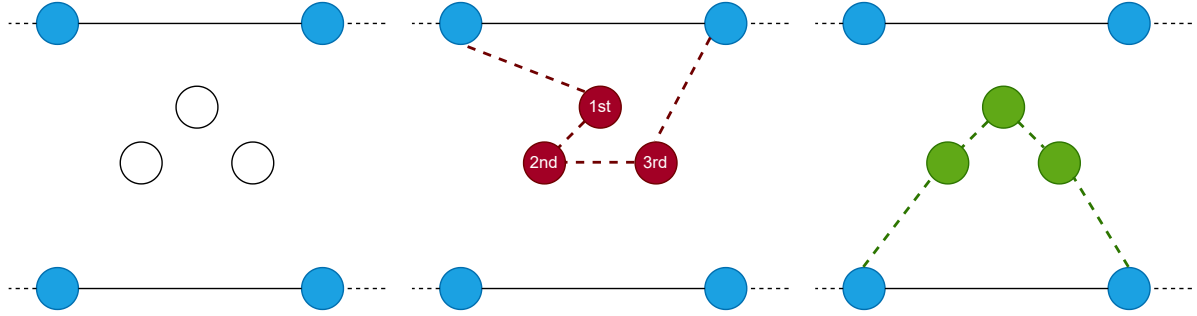


Figure 18: Base Problem

Figure 19: Incorrect Solution

Figure 20: Correct Solution

In Figure18, the base problem involves a partial route that passes through the top and bottom of a cluster of three nodes in the centre. CIH will construct the local optimal route(Figure19) to the cluster in the order shown in the diagram based on picking local optimal options using the cost function(Equation2). This is because the topmost node is closest to the route and therefore has the lowest cost, whereas picking one of the other suboptimal nodes would result in the global optimal solution(Figure20).

4.2 Static Look-Ahead Cheapest Insertion Heuristic(SLACIH)

This report explains how a look-ahead search of static depth is applied to CIH. To help avoid pitfalls, a look-ahead design can be employed to guide decision-making^[79]. Applying look-ahead to the Cheapest Insertion Heuristic(CIH) cannot be done in the same manner as the Look-Ahead Nearest Neighbour^[80] (Section3.3.1); this is due to the nearest neighbour using a sequential head as a point of insertion, while CIH evaluates every node to be added anywhere in the route.

This report examines a new algorithm, the Static Look-Ahead Cheapest Insertion Heuristic(SLACIH), or look-ahead insertion for short. If a specific look-ahead depth is specified, it can be listed in brackets after the name, e.g. SLACIH(3) for a depth of 3.

Designing an effective and working look-ahead algorithm requires the heuristic to create a wide variation in branching solutions, enabling efficient foresight to be achieved. Taking the intuitive approach of doing sequential CIH iterations will not be effective. This is because

the probability of finding unique and relevant solutions is low, given that the CIH iterations are unrelated to past insertions. To solve this, a temporary path which contains only past insertions is used exclusively as the search space for evaluating CIH in the look-ahead. This divides the SLACIH search into two subprocesses:

- Normal CIH search to find where it would be added to the partial path
- Temporary look-ahead search to find which nodes would be added next based on previous insertions

This preserves the properties of the CIH cost function(Equation2), allowing nodes to be added anywhere in the route, provided that the look-ahead only considers options that are relevant. The steps of SLACIH are as follows:

1. Generate an initial starting partial set
2. For each unvisited node:
 - 2.1. Calculate the cost function of adding the node to every position in the partial route
 - 2.2. Add the unvisited node and the two visited nodes found in 2.1 to a temporary route
 - 2.3. Set a new depth limit if the depth exceeds the number of unvisited nodes
 - 2.4. For each unvisited node:
 - 2.4.1. Calculate the cost function of adding the node to every position in the temporary route
 - 2.4.2. Add the cheapest node to the temporary
 - 2.4.3. Increases depth count by one
 - 2.4.4. Repeat step 2.4 until the desired look-ahead depth
3. Calculate the temporary route length and compare against best known
4. Add the original node to the position found in step 2
5. Repeat step 2 until there are no unvisited nodes

When comparing it to CIH, steps 1, 2.1, 4, and 5 are identical to those in CIH, with steps 2.2 to 3 forming part of the look-ahead design. The look-ahead design considers every node, n ,

against a partial route of size $depth + 2$, for $depth$ iterations. Counting the nested For loops, the time complexity can be calculated:

$$O(\underbrace{n^2 \cdot (n)}_{\text{CIH}} + \underbrace{n \cdot (depth + 2) \cdot depth}_{\text{Look-ahead}})) \quad (4)$$

SLACHI Time Complexity

But can be simplified to $O(n^5)$:

$$\begin{aligned} O(n^2 \cdot (n + n \cdot (depth + 2) \cdot depth)) & \quad // \text{ Base equation} \\ O(n^2 \cdot (n + n \cdot depth^2)) & \quad // \text{ Simplify and ignore +2} \\ O(n^3 + n^3 \cdot depth^2) & \quad // \text{ Expand brackets} \\ O(n^3 + n^3 \cdot n^2) & \quad // \text{ Worse-case value of depth} \\ O(n^3 + n^5) & \quad // \text{ Simplify} \\ O(n^5) & \quad // \text{ Ignore lower-order terms} \end{aligned}$$

Look-ahead depth can have values $n \geq depth \geq 0$, allowing it to be applied to n in Big O notation. It also shows that the time complexity is bound to a quintic polynomial(5th power), regardless of the depth of the look-ahead.

Visually, the logic of evaluating temporary routes can be illustrated in geometric cases for each iteration. Blue nodes represent visited nodes, white nodes represent unvisited nodes, the orange node represents the node being considered, and yellow lines and nodes represent the temporary route.

Inner Working

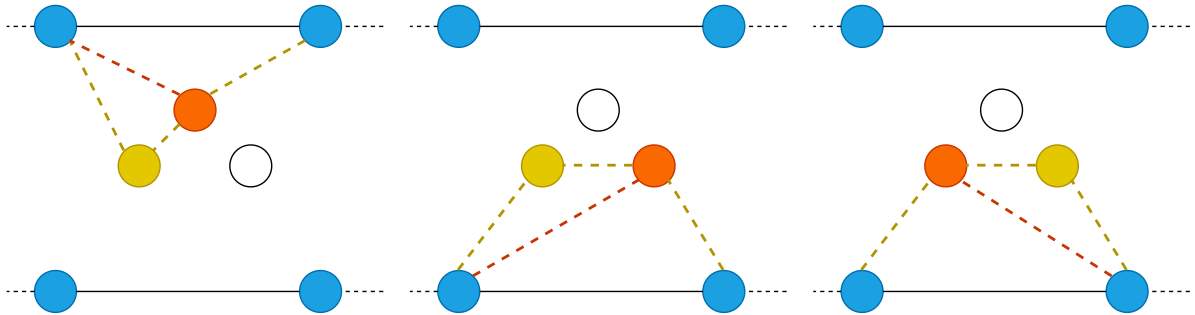


Figure 21: Node 1

Figure 22: Node 2

Figure 23: Node 3

Figures 21-23 show SLACIH(1) with a look-ahead depth of one, evaluating each unvisited node with foresight in yellow. Unlike CIH(Figure 19), which incorrectly considers the northernmost node due to its close proximity to the route, SLACIH calculates the cost of adding the node and the next.

Outer Working

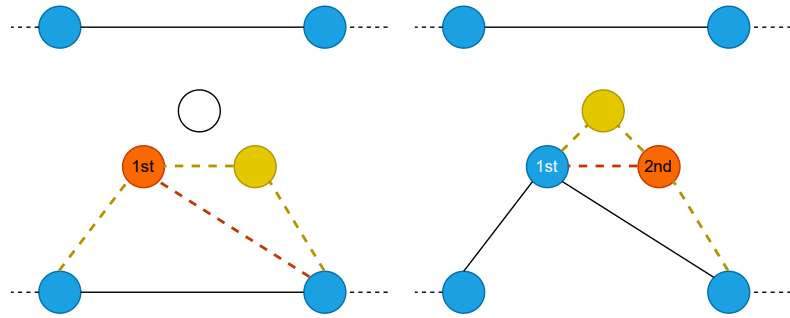


Figure 24: Iteration 1

Figure 25: Iteration 2

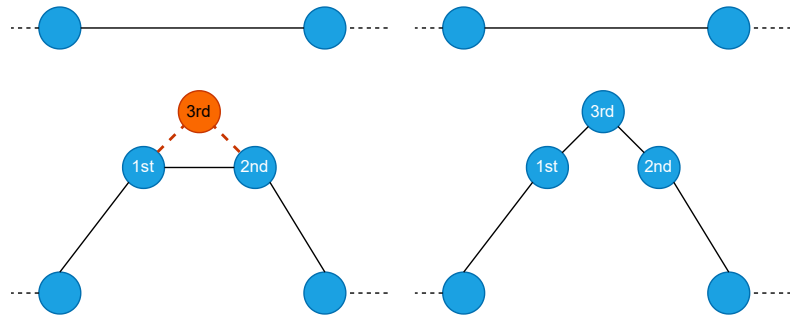


Figure 26: Iteration 3

Figure 27: Solution

Figures 24-27 show SLACIH(1) with a look-ahead depth of one, iteratively adding the next node to the route based on its evaluation of the cheapest temporary route found during the inner workings. In Figure 26, there is no look-ahead (yellow lines) because there are no more unvisited nodes to visit. Therefore, the look-ahead depth is defined by:

$$\text{depth} = \min(\text{depth}, \# \text{ of unvisited nodes}) = \min(\text{depth}, |\text{graph}| - |\text{route}| - 1) \quad (5)$$

4.2.1 Pseudocode

Algorithm 3 SLACIH($depth$: integer, g : Graph)

```

1:  $path :=$  initial partial set ▷ Random node or Convex Hull(Step1.)
2: while  $\exists$  node  $\notin$  path do ▷ Step5.
3:    $[b\_node, b\_position, b\_cost] :=$  default values
4:   for node  $\notin$  path do ▷ Step2.
5:      $[node, position, cost] =$  CIH-Search( $depth, node, path, g$ )
6:     if  $cost < b\_cost$  then ▷ Step3.
7:        $[b\_node, b\_position, b\_cost] = [node, position, cost]$ 
8:   path.add( $b\_node, b\_position$ ) ▷ Step4.

```

Algorithm 4 CIH-Search($depth$: integer, n : Node, $path$: route, g : graph)

```

1:  $[b\_position, b\_cost] :=$  default values
2: for node  $\in$  path do ▷ Step2.1.
3:    $cost :=$  Distance( $n, node_i, node_{i+1}$ )
4:   if  $cost < b\_cost$  then
5:      $[b\_position, b\_cost] = [i, cost]$ 
6:
7:  $temporary\_list := \{node_i, n, node_{i+1}\}$  ▷ Step2.2.
8:  $depth := \min(depth, |g| - |path| - 1)$  ▷ Step2.3.
9:
10:  $b\_cost :=$  lookahead-search( $depth, temporary\_path, path, g$ )
11: Return  $[b\_position, b\_cost]$ 

```

Algorithm 5 lookahead-search($depth$: integer, $temporary_path$: path, $path$: path, g : graph)

```

1: if  $depth < 0$  then
2:   Return 0
3:  $[b\_node, b\_position, b\_cost] :=$  default values
4: for node  $\in g \ni temporary\_path, path$  do ▷ Step2.4.
5:   for  $p \in temporary\_path$  do
6:      $cost :=$  Distance( $node, p_i, p_{i+1}$ ) ▷ Step2.4.1.
7:     if  $cost < b\_cost$  then
8:        $[b\_node, b\_position, b\_cost] = [node, i, cost]$ 
9:   temporary_path.add( $b\_node, b\_position$ ) ▷ Step2.4.2.
10:  $depth = depth - 1$  ▷ Step2.4.3.
11: Return lookahead-search( $depth - 1, temporary\_path, path, g$ ) ▷ Step2.4.4.

```

4.3 Pitfalls With New Solution

Pitfalls are specific cases where an algorithm makes the wrong choice; all heuristics have them, including look-ahead design. While look-ahead is valuable in identifying local pitfalls, due to the greedy nature of CIH, it is not enough to prevent it from making mistakes(Figure18). SLACIH remains a greedy heuristic that can fail under certain circumstances, as demonstrated in Section4.1. This is due to the reliance on a cost function and local greedy optimisation remaining as the decision-making process at each iteration of the look-ahead.

This can be proven by contradiction by applying $SLACIH(n)$ to a problem and observing that it fails to achieve the exact solution. If $SLACIH(n)$ provided an exact solution for every TSP problem, then the $P=NP$ problem would be solved because $SLACIH(n)$ has a polynomial time complexity of $O(n^5)$.

SLACIH introduces another pitfall unique to look-ahead algorithms. The depth of look-ahead searches can impact the decision-making process when considering the required partial route size(depth) as the cost function. This can be visualised in geometric space with a small cluster of nodes being considered by $SLACIH(3)$, a look-ahead depth of 3.

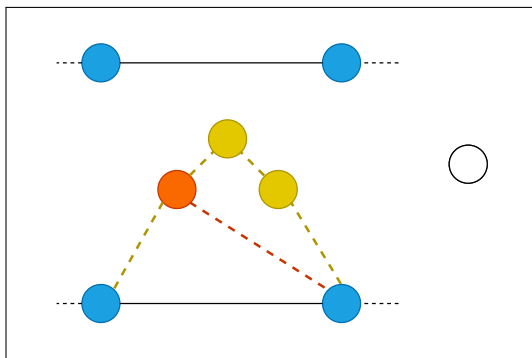


Figure 28: SLACIH(2)

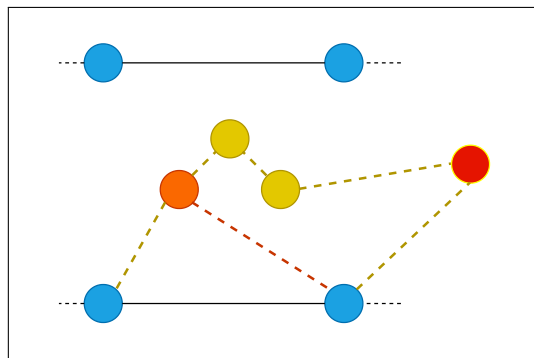


Figure 29: SLACIH(3)

When considering adding other nodes to the example, they can be far away(high cost) from a small cluster(Figure28). If a look-ahead depth is larger than the size of the cluster, it can unfairly evaluate the cost of picking certain nodes(Figure29), leading to worse foresight predictions than those actually represented and causing local minima to form. This is not a problem that can be solved, as determining which nodes to include or exclude is equivalent to the TSP; therefore, it cannot be solved with a polynomial-time algorithm. It also proves the case that having higher look-ahead depth does not guarantee a better solution. However, by using statistics and logic, refining the look-ahead search depth can lead to improvements, as seen in Section4.4.

4.4 Dynamic Look-Ahead Cheapest Insertion Heuristic(DLACIH)

Dynamically changing the look-ahead depth can combat the pitfalls that a static look-ahead design introduces. Whereas static look-ahead maintains the same look-ahead depth throughout the algorithm, dynamic look-ahead adjusts the look-ahead depth at runtime based on available information. This information could include problem size, the number of unvisited nodes, the number of nodes within an area, or patterns in past insertions.

The benefit of dynamic(sometimes called adaptive) look-ahead has been demonstrated in other research fields(mostly machine learning)^[102] due to its improved results and runtime compared to static models. This is possible because it uses already known information to make predictions when look-ahead is or is not required^[103].

This report examines a variety of new algorithms, the Dynamic Look-Ahead Cheapest Insertion Heuristic(DLACIH), or dynamic look-ahead insertion for short. There are numerous approaches to the behaviour exhibited by dynamic depth; this report proposes multiple new designs, but only the static dynamic in Section4.4.1 is implemented.

4.4.1 Static Dynamic

As discussed in Section4.3, a higher look-ahead depth is not always best, but neither is a lower depth. Using statistics, it is determined that a given problem size has an optimal look-ahead depth. Using a formula(6) to map the trends, the depth can be dynamically set at run time.

$$depth = \min(max_depth, \max(0, \text{Round}(\overset{\text{Multiplier}}{\boxed{M}} \cdot \ln(\overset{\text{Logarithmic}}{\boxed{L \cdot size}}) - \overset{\text{Constant}}{\boxed{C}}))) \quad (6)$$

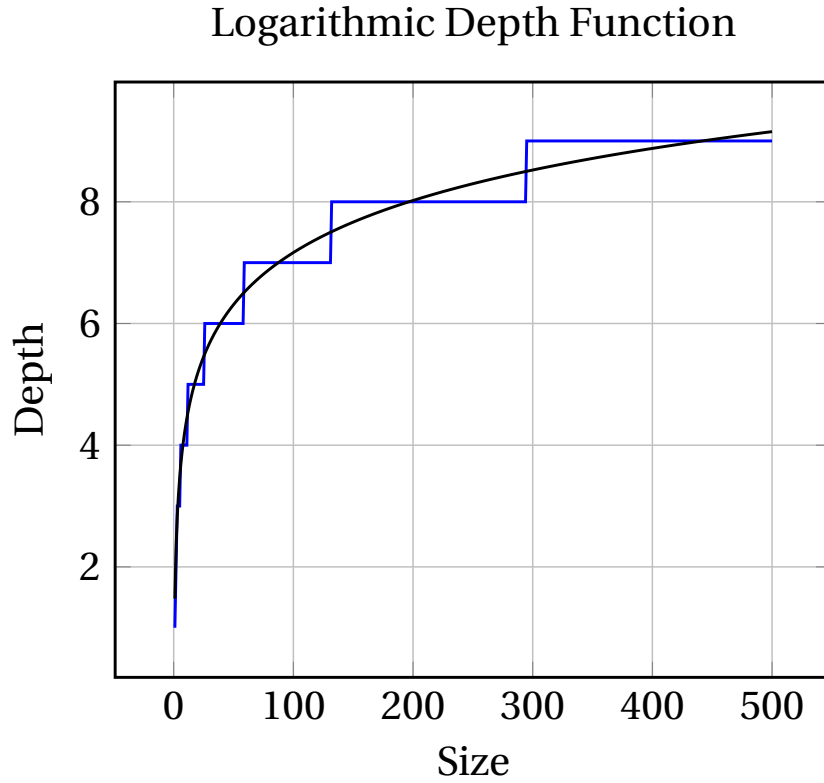


Figure 30: Plot Of Depth Function

The formula states that the depth can be set to any logarithmic curve with respect to size, as defined by three variables (M = multiplication, L = logarithmic, C = constant), provided that it satisfies $max_depth \geq depth \geq 0$. The values of M , L , and C must be set before run-time but can be optimised from testing and applying gradient descent. There are two ways the formula can be used to set the depth:

- Beginning: Set the optimal depth at the start for the whole length of the algorithm
- Iteratively: Set the optimal depth each iteration of DLACIH

Setting it at the beginning means DLACIH will behave like SLACIH, but it guarantees that it will produce a sufficiently good result. Setting it in each iteration alters the design of DLACIH from SLACIH, enabling finer tuning and further research into optimal curves. As the algorithm progresses (high sparsity), more nodes are added to the route, and the proximity of clusters to multiple route nodes increases (low sparsity). Thus, a lower look-ahead depth is more effective towards the end, whereas a higher look-ahead depth is more beneficial at the beginning.

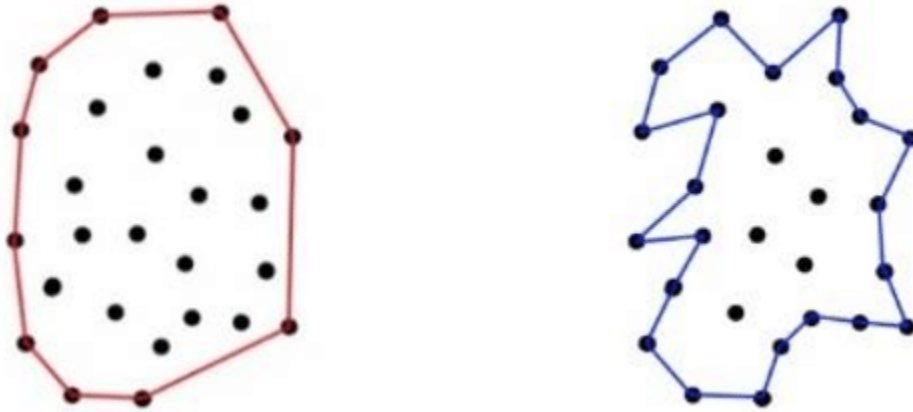


Figure 31: High(Left) vs Low(Right) Sparsity
[104]

4.4.2 Adaptive Dynamic

Instead of applying statistics(Section4.4.1), a logical step can be added to aid in the dynamic look-ahead process^[103]. This typically involves an evaluation of the previous iteration(s)^[102] or information gathering^[103].

When evaluating previous iterations, an approach similar to the Lin-Kernighan adaptive selection^[81] can be used(Figure32). Changing the depth of look-ahead is beneficial in avoiding local minima that are present within a certain range of depths. By adjusting the depth(Equation7), the heuristic can proceed with more promising additions that would have been hindered by pitfalls otherwise.

$$depth = \min(max_depth, \max(0, M \cdot \ln(L \cdot size) - C + \overset{\text{Amplitude}}{\boxed{D}} \cdot \sin(\overset{\text{Frequency}}{\boxed{F \cdot \#iteration}}))) \quad (7)$$

If acting upon information gathered from the current state, logical assumptions can be used to define the depth. Adding a step that adaptively changes the depth based on the proximity to other nodes can help avoid poor representation in the cost function. A crude method for defining closeness is to count the number of nodes within a specified distance of the node. The closeness cut-off can be defined based on an arbitrary size(Figure33), statistical assumptions of the problem(such as the mean value), or the best efforts of previous iterations.

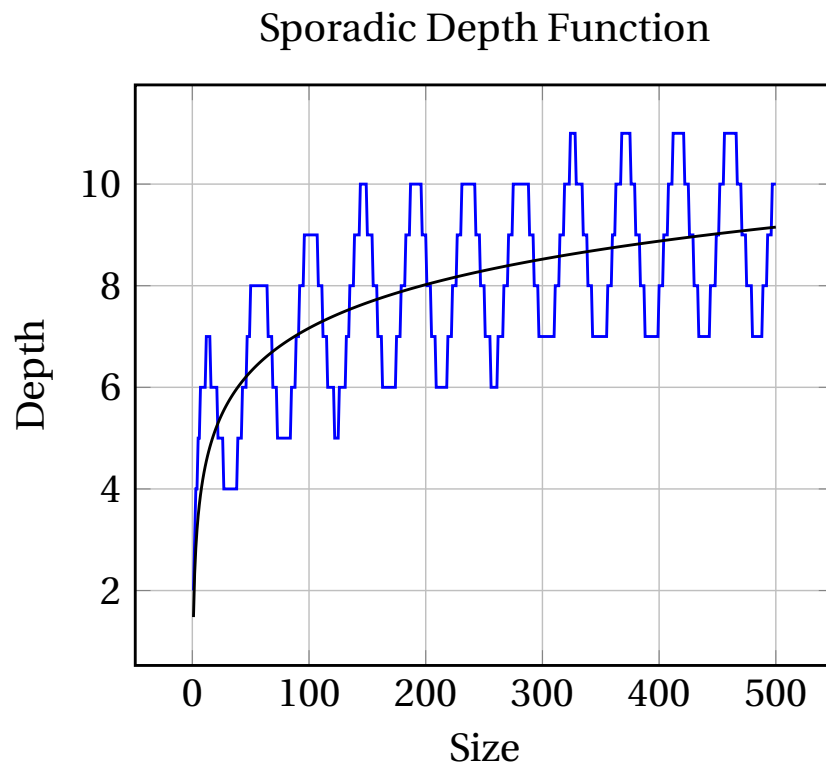


Figure 32: Plot Of Sporadic Depth Function

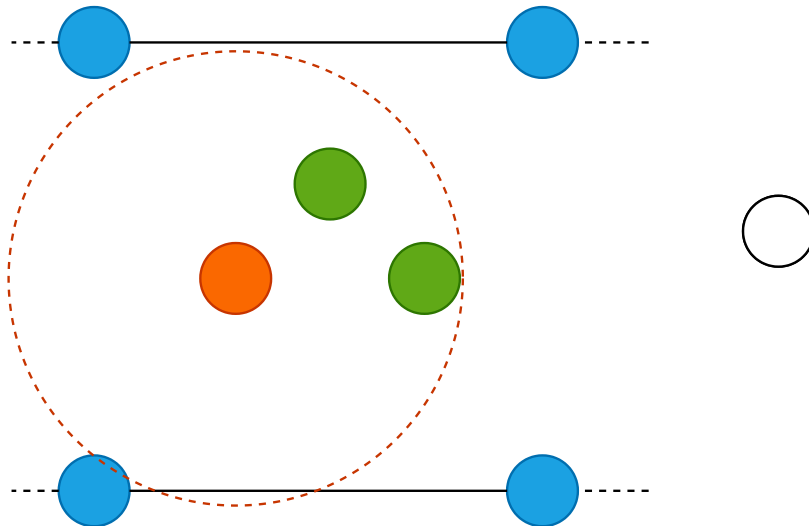


Figure 33: Closeness Search

5 Methodology

An organised approach to the evaluation and testing is required to study the proposed algorithms analytically. The Travelling Salesman Problem is a single-objective optimisation problem^[105] with the sole goal of minimising the route cost(Equation1). However, it is not the only metric worth considering when evaluating heuristics, as runtime is a major factor in determining which algorithm to use^[106]. Therefore, the data collection method must remain unbiased for the comparisons to have validity.

Empirical research is a research method that involves observing the direct result of an action and gathering data^[107]. The values observed are considered empirical data, which can be quantitatively analysed to aid in decision making and reaching a conclusion^[107]. Due to the nature of NP-hard problems, it is improbable^[105] to test heuristics against random exact solutions. Therefore, comparing them to other heuristics is an appropriate approach; this is called blind(statistical) testing^[108].

5.1 Empirical Testing

Using large data collection and data analysis, trends and properties are outlined in the testing, which would otherwise be obscured by randomness^[108]. Much like real-life experiments, the collected data(dependent variable) must have controlled variables and a single independent variable. The independent variable is the algorithm and its unique parameters, thus SLACIH(1) and SLACIH(2) are two different independent variable entries. The control is the identical Travelling Salesman Problem and system properties executed for each algorithm. A set of experiments with the same control variable is referred to as an instance. Generating multiple random instances creates a data table in a process known as Instances vs Algorithm^[108]. In the table, all instances must have the same TSP properties, e.g. problem size and random generation method.

	Algorithm 1	Algorithm 2	...	Algorithm n
Instance 1				
Instance 2				
⋮				
Instance m				

Figure 34: Instances Vs Algorithm
^[106]

Once the table has been populated with a sufficient number of instances, to reduce randomness,

data analysis can be used to draw conclusions. Statistical properties, such as averages, standard deviation, and inference, are explored as avenues for comparison.

5.2 TSPLib

In the research for the Travelling Salesman Problem, there is a library of tests called TSPLib^[109], which contains famous real-world examples and common pitfall problems. It contains 130 problems of varying graph sizes, with the largest having 85,900 nodes. All have been solved with an exact solution by the Concorde TSP Solver(Section3.1.1), making TSPLib a suitable library for benchmarking heuristics against the global optimal^[110]. TSPLib is also useful as a ground truth for comparing research papers due to its ubiquity as a benchmark.

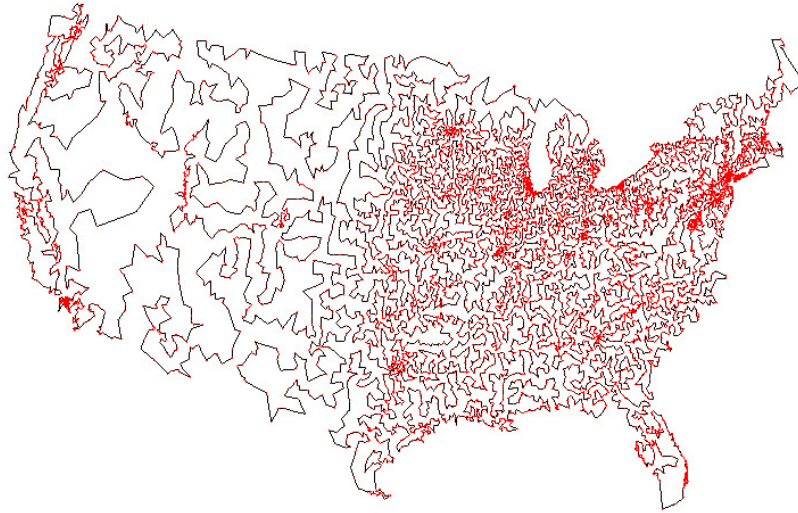


Figure 35: TSPLib: USA13509
[111]

As the problem solutions vary in cost, taking the difference from the optimal solution as a percentage(Equation8) allows for calculating the average performance of a heuristic across the entire TSPLib(Equation9). This normalisation allows for each problem to contribute evenly to a heuristic score.

$$p_{solution} = \frac{solution_{heuristic} - solution_{optimal}}{solution_{optimal}} \quad (8)$$

$$p_{TSPLib} = \frac{\sum p_{solution}}{|p_{solution}|} \quad (9)$$

6 System Requirements

Until this point, the report has primarily focused on research and theoretical designs, but software development is a significant aspect required to prove and analyse the theory. Making the right system design choices is crucial to the success of the project's aims and goals, ensuring that valuable time is not wasted on refactoring and bug fixing. Multiple working computer programs are required to fill all the project needs; therefore, the code should be adaptable and highly interoperable. The programs should utilise state-of-the-art technology to leverage all the benefits to achieve the highest success. Planning and system requirements are crucial, as most IT projects fail because there were not clearly defined requirements^[112].

6.1 Functional Requirements

Functional requirements define what the product(s) should do regarding behaviour and features; achieving these requirements is essential for the project's success. Each requirement can be classified by type, description, and aim in Table2.

<u>Name</u>	<u>Type</u>	<u>Description</u>	<u>Aim</u>
1.a) Graph Theory	Data	Store vertices and edges	The program should be able to handle storing any number of vertices and edges in memory as mentioned in Section1.2
1.b) TSP Constraints	Logic	Follow the rules of a TSP problem	The program should not break any of the TSP constraints mentioned in Section1.2
1.c) Euclidean TSP	Logic	Follow the rules of a Euclidean distance	The program should calculate the Euclidean distance correctly between two vertices
2.a) Facilities for Algorithms	Logic	Allow multiple algorithms to be used	The program should be able to be used for numerous algorithms
2.b) Benchmarking Algorithm	System	Benchmarking of route length and time taken	The program should accurately report and store the results of an algorithm execution
2.c) Algorithm Fairness	Performance	All algorithms should operate at their best	The program should not negatively impact the performance of an algorithm
2.d) Testing Suite	System	Allow for multiple different types of tests	The program should be able to run various types of tests with many parameters repetitively and for long periods
3.a) Display TSP Solutions	UI/UX	GUI output of the algorithms solutions	The program should adequately display graphs and Hamiltonian cycles
4.a) Collect Test Data	Data	Collect multiple instances of test data	The program should use secondary storage, a proper text-based formatting system
4.b) Analysis Test Data	System	Produce statistics and graphs	The program should provide data analysis on the collected data to produce meaningful statistical insight
5.a) Unit Test	System	Adequately unit test code base	The program should have unit test coverage of the core shared library codebase, focusing on systems used by every component

Table 2: Functional Requirements

6.2 Non-Functional Requirements

Non-functional requirements define the level of quality the product(s) should have in terms of behaviour and features; achieving these requirements is not essential for the project's success but would be beneficial. Each requirement can be classified by its aim and type in a table.

<u>Aim</u>	<u>Type</u>
The user should be able to select the type of algorithm they want easily	Usability
The program should never fail or crash	Reliability
The program should be responsive and utilise all computing power	Performance
The program should be adaptable and fill the needs of each executable	Scalability
The GUI should be easy to read for all users, e.g. inclusive of those with colour blindness	Usability

Table 3: Non-Functional Requirements

6.3 C++

C++ is a low-level programming language first created in 1985 by Bjarne Stroustrup^[113]. It is widely used in algorithmic research and industry due to its high performance and low-level capabilities. In this project, C++ was selected as the primary language due to prior experience and previous personal projects involving the language. MSVC^[114] compiler was used as it integrates seamlessly with Visual Studio and C++20^[115] for it is the most modern long-term support(LTS) version.

C++ has no built-in memory management and relies on the programmer to handle allocation and deletion. This removes the overhead of an automatic memory management system but can lead to data corruption or memory violations. The benefit of allowing the programmer to decide is the ability to use either stack vs heap memory.

- Stack: Faster but limited by size and must be known at compile time
- Heap: Unlimited size and can be dynamically allocated, but slower as it requires OS system call and uses a pointer to access.

Accessing memory is a leading cause of slow performance times due to cache misses^[116]. C++ has low-level calls that are unique to low-level languages, allowing for direct manipulation of

computer memory. This allows for logical memory tricks and greater control over memory to accommodate each system's architecture^[117]. Both must be used when programming in C++, but stack memory is favourable in most situations.

Object-Oriented Programming(OOP) has become the standard programming paradigm for most use cases. OOP is dynamic as encapsulation and polymorphic properties can be applied to most problems. The abstract nature and use of Creational Patterns lend OOP well to this project for initiating multiple algorithms.

6.3.1 API & Interfaces

To satisfy the multiple functional requirements(Table2), various computer programs(executables) will need to be developed to achieve each of the different requirements. Each program will require the same features and data, so to avoid bugs and repetitive code; core functionality is placed in a library. There are two types of libraries in C++:

- Static Library: Library code is added to the source code and compiled at compile time
- Dynamic Library: Library code is compiled into a DLL and linked at run-time

The benefits and differences depend on a few factors, but the static library was preferred due to template metaprogramming(Section6.3.2). However, both remove the dependency of an owner, and a static library acts as the project's base. To ensure practical software development, it is still essential to understand how the library functions and interacts with each program; this is achieved by following standard Design Patterns, APIs, and Interfaces.

As seen in Section3, several stages and optimisations can be applied to create an overall heuristic, including tour constructors, tour optimisers, partitioning systems, and caching modes. To construct any number of combinations by hand would be infeasible, so a Creational Pattern in OOP, called a Builder^[118], is used like a cookie cutter for each option. The idea is to have a single Builder class that utilises composition to construct the algorithm for the possibilities it provides. This requires interfaces for the builder class to call upon its objects.

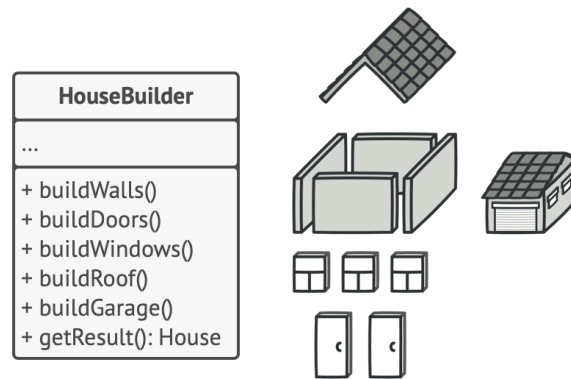


Figure 36: Builder Pattern
[118]

Interfaces are public abstract classes that utilise the polymorphic nature. Abstract classes cannot exist as objects due to being incomplete classes, but they can be inherited by child classes to gain their properties. This ensures that each built object has the correct interface for immutability within the larger system.

To ensure the user(programmer) does not interact with the library incorrectly, APIs are used to limit the control they have over the underlying data. API's also help programmers more easily understand what the code is doing and what the original creator had in mind. APIs are heavily relied upon for communication between the library and each executable.

6.3.2 Template Metaprogramming

Template Metaprogramming(TMP) is an additional step in the C++ compiler that introduces compile-time polymorphism by utilising template functions^[119]. It allows a single template function to have infinite copies of itself without requiring additional code. Template functions require a template variable, which takes the values of data types, functions, or constant values. At compile time, the template function variables are instantiated in the template functions to create inlined generic source code.

Listing 1: Template Code

```

1 template<typename var>
2 void foo(var a) {
3     std::cout << "value : " << a << "\n";
4 }
5

```

```

6  /*
7  What the compiler would generate:
8
9  void foo<int>(int a) {
10     std::cout << "value : " << a << "\n";
11 }
12
13 void foo<std::string>(std::string a) {
14     std::cout << "value : " << a << "\n";
15 }
16 */
17
18 template<int var>
19 class goo {
20 public:
21     static void bar() {
22         std::cout << "value : " << std::to_string(var) << "\n";
23     }
24 };
25
26 /*
27 What the compiler would generate
28
29 static void goo<5>::bar() {
30     std::cout << "value : " << std::to_string(5) << "\n";
31 }
32 */
33
34 int main()
35 {
36     foo<int>(10);           // print "value : 10"
37     foo<std::string>("hello"); // print "value : hello"
38
39     goo<5>::bar();         // print "value : 5"
40 }

```

In the example, Listing1, *foo()* is a template function, and *var* is a template function variable.

In the *main()* function, *foo* is instantiated with the data types *int* and *string*, and then arguments of those types are passed. The program compiles and runs as expected, with the *foo <int> ()* and *foo <string> ()* functions being generated before compile time, as represented by the code in the comment (Line 9). Classes can also have templates, which can define member fields and functions (Listing 2).

Listing 2: Advance Template Code

```
1 class abstract { // Pure abstract class
2     virtual void foo() = 0;
3 };
4
5 class child : public abstract { // Abstract child class
6     void foo() override {
7         std::cout << "real function";
8     }
9 };
10
11 template<typename T>
12 class real {
13     static_assert(std::is_base_of<abstract, T>::value, "T must derive from abstract");
14
15     T var; // Compile-Time Polymorphism: In-line real class without V-Table
16 };
17
18 int main() {
19     real<child> b; // Abstract class on the stack
20 }
```

Metaprogramming has its roots in compiler development but is heavily applied in performance optimisation code. This is because a constant expression can be evaluated at compile time, leading to the removal (inlining) of branching and expression evaluation. Much like its name, compile-time polymorphism, metaprogramming^[120] can mimic OOP polymorphism at compile time (Listing 2). This is faster than normal polymorphism, as it eliminates the V-Table, which determines which child virtual function to invoke. Calling the V-Table incurs a constant time slowdown, which is undesirable when performance is a concern and the number of calls varies between TSP algorithms.

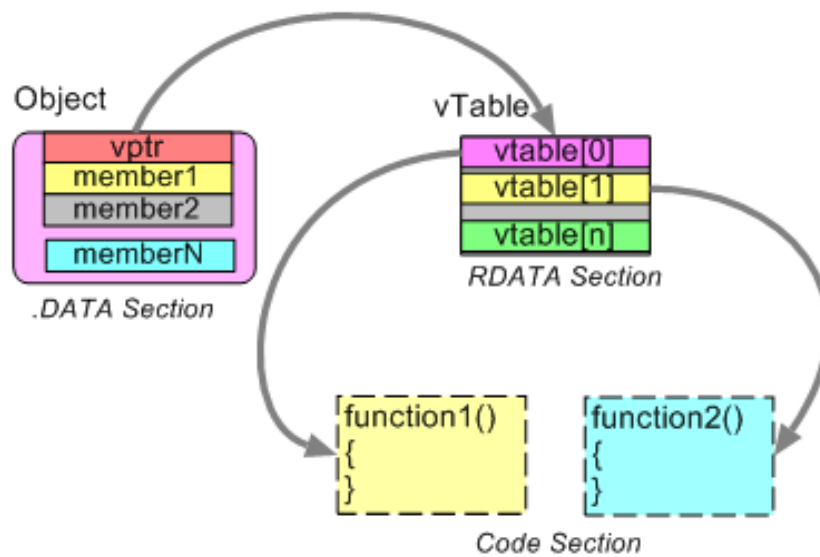


Figure 37: C++ V-Table
[118]

Another performance benefit of TMP is compile instantiation. Many data types must be placed on the heap due to their dynamic size, which incurs a performance hit with each call. As TMP instantiates classes with the type in place at compile time, it can allocate a static size on the stack for abstract child classes, thereby avoiding any performance hit (Listing2).

As this project is heavily performance-focused, fairness is necessary; not using templates would be a mistake. Compile-time polymorphism enables builder classes with constant slowdowns to behave like a monolithic class, allowing in-line and optimisation of function calls. Templates have their disadvantages, including steep learning curves, reduced flexibility, additional complexity, difficult to debug, long compilation times, and memory usage issues; however, these are negligible for the scale and scope of this project.

6.3.3 QT6

QT^[121] is a cross-platform GUI library for creating graphical applications natively on all operating systems and embedded hardware. It is written in C++ but can also be developed with Python. QT6 is the most modern iteration, having undergone 29 years of development and featuring an open-source license, making it a go-to choice for research and non-profit developers. The needs of this project do not necessitate a wide use of graphical features, which is why choosing QT6, with its extensive catalogue of tutorials, was ideal.

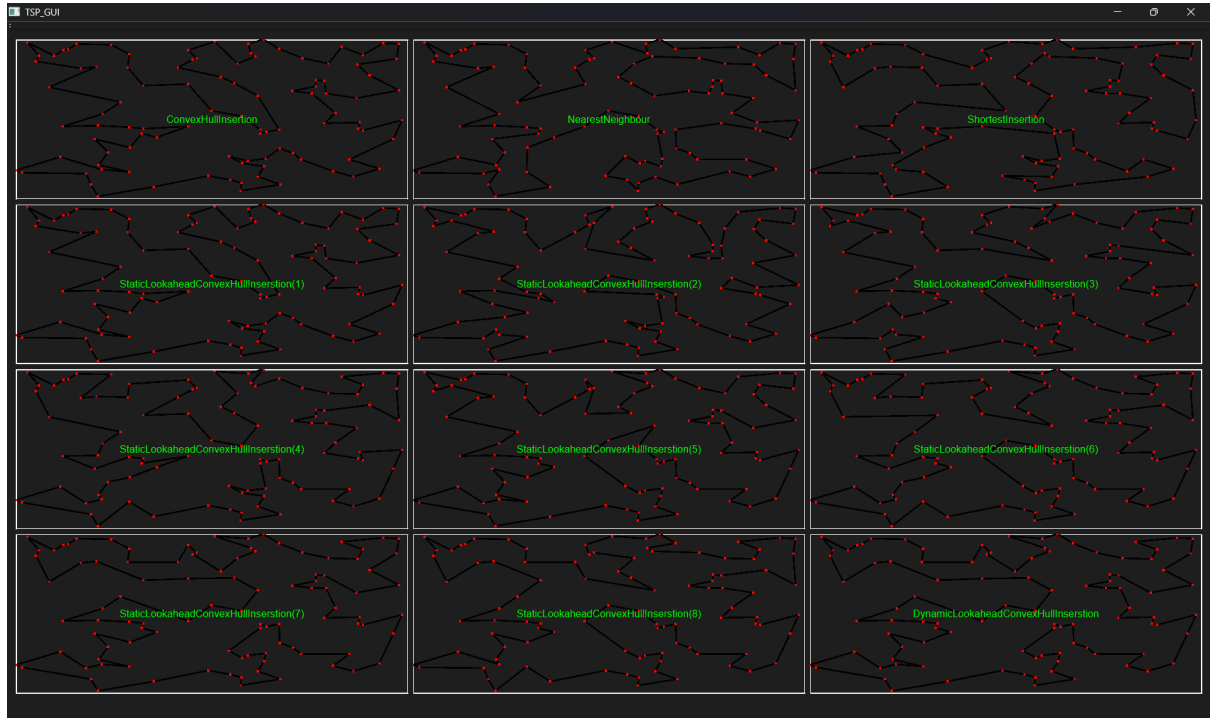


Figure 38: Example: QT6 Application GUI

6.3.4 C++ Libraries

Libraries are valuable resources for obtaining a pre-made solution to complex functionality that developers widely encounter. However, when C++ was being developed, library support and development were not as prolific as modern languages. For this project, the `libxlsxwriter` library^[122] helped generate Excel files. GNU Scientific Library^[123] was used for the gradient descent function in optimising the dynamic look-ahead arguments.

6.4 Hardware

Hardware is a crucial factor in testing NP-hard problems; from tiny microcontrollers to massive supercomputers, the amount of available computing power determines the problem size and the type of algorithm that can be used. The project was completed using a home desktop PC running Windows 11, equipped with an Intel i9-13900 processor and 32GB of RAM.

6.4.1 Consistency

For the fair testing of algorithms(Section5), physical and external factors need to be controlled as a control variable to prevent bias. Physical factors comprise of power draw to thermal temperatures, while external factors include background tasks and execution mode. With no funding or external resources, controlling many of the variables was hindered.

Thermal temperatures of the CPU constitute the most significant downfall; excessive heat can trigger thermal throttling, reducing performance. Managing thermal temperatures with effective cooling systems and maintaining a consistent ambient temperature was not possible with the available setup. Therefore, overclocking the CPU was disabled to maintain a stable clock rate for extended periods.

When compiling the program, the identical execution flags and release mode should be constant. Tests should not be run quickly after boot or with background tasks running, as these may consume CPU time.

6.4.2 Multi-Threading

To fulfil the non-functional requirements(Table3) and reduce the time required to gather all the data, tests are run in parallel. Each process also uses asynchronous timeout calls to prevent tests from running indefinitely. Therefore, a multi-core processor is a requirement for achieving stable performance.

6.4.3 Memory

The Travelling Salesman Problem and look-ahead design can consume a significant amount of memory if the problem is sufficiently large. With modern hardware storing GB of RAM and MB of cache, the space capacity is not likely to be a limiting factor. However, the constant reading and writing of memory can hinder performance; therefore, quicker RAM is favourable.

7 Design

Breaking the problem down into building blocks of abstract classes and interfaces allows each system to define how it should interact and behave. As mentioned in Section6.3.1, creational patterns are valuable techniques for constructing large-scale systems. Combining Builder classes and API is the basis for how the system is structured.

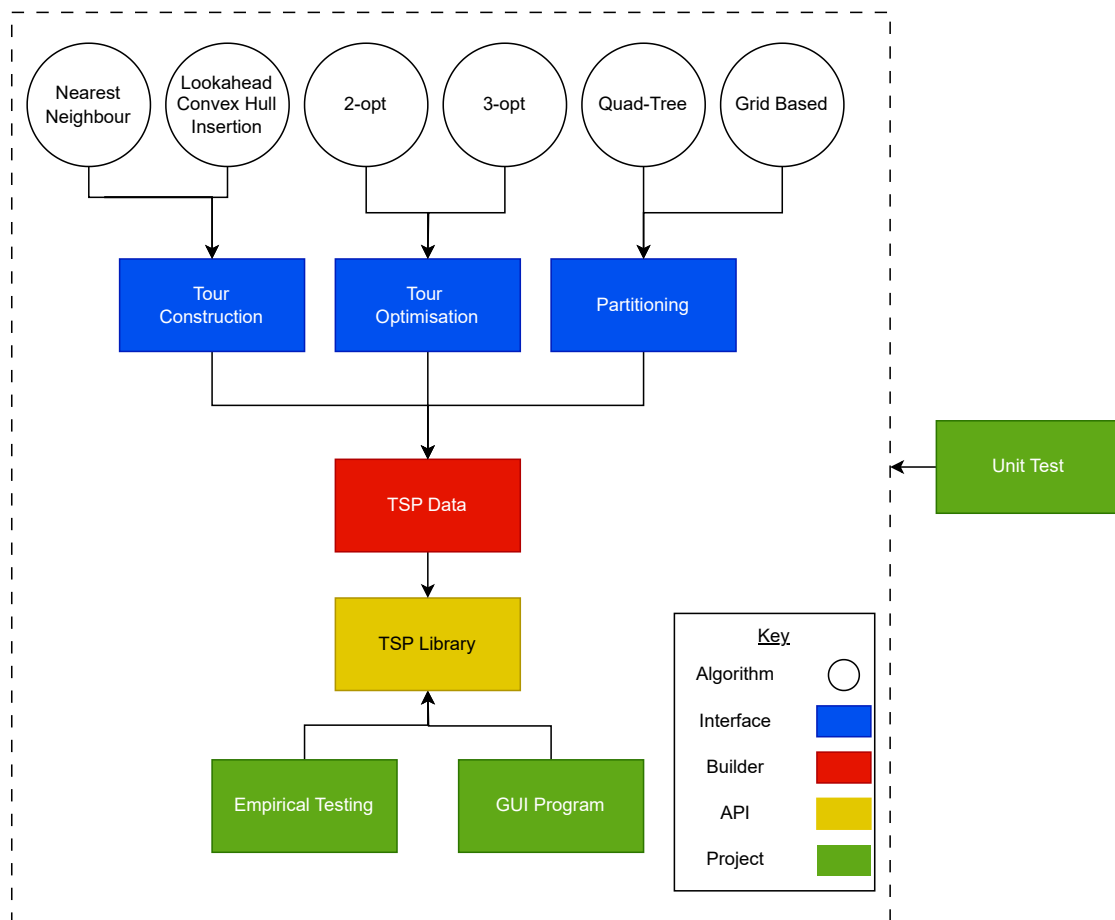


Figure 39: Class Structure

Figure39 illustrates the class hierarchy and the interactions between each system block for the core static library code. At each level, a layer of abstraction is applied to hide information about the other blocks, allowing them to work independently.

7.1 TSP Library API

TSP_Library
+ m_data: TSP_Data
+ TSP_Library(Seed): Void
+ Run(TSP_Args): TSP_File

Figure 40: TSP Lib

The TSP Library serves as the entry point for the static library, providing the API that controls the creation and execution of TSP algorithms through the TSP_data class. It is responsible for parsing the data and serialising the result. It houses all the asynchronous timeout code and benchmarking functionality.

There are only two functions: the constructor and the run function. The constructor uses a seed value to generate m_data as a composition object with all the data required for the functionality. The run function controls the handling of timeouts, the execution of the algorithm steps, and data serialisation to the TSP_File type. The ordering of the run function is as follows:

1. Start benchmark timer
2. Initialise partitioning data
3. Initialise caching data
4. Run tour constructor algorithm
5. Run tour optimiser algorithm
6. Stop benchmark timer
7. Validate the route
8. Process data and serialise to TSP_File

7.1.1 TSP File

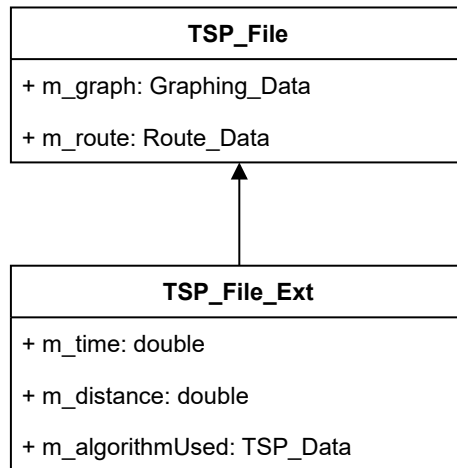


Figure 41: TSP File

TSP_File represents multiple classes for deserialising and serialising the '.TSP' file type, which originates from the TSPLib format. The original file type contains relevant information to reconstruct the problem and its solution. For this project, the file type was expanded using polymorphism by adding benchmarking statistics with the class TSP_File_Ext.

7.2 TSP Data API

TSP_Data
+ m_data: Graphing_Data + m_route: Route_Data + m_cache: Caching_Mode + m_partitioning: Partitioning + m_constructor: Constructor + m_optimiser: Optimiser
+ TSP_Data(Seed): Void + getDistance(Point, Point) : Double + getPointsInArea(Area) : List<Point> + getRoute() : Route_Data + addPointInRoute(Point,Position) : Void + swapRoutePoints(Point,Point) : Void

Figure 42: TSP Data

TSP_Data is what connects everything together and ensures the TSP properties are kept. It is a builder class(Section6.3.1) for generating any variation of TSP algorithm from the number of TSP Modules(Section7.3). It stores the graph theory and route creation data and logic, serving as an API(Table4) for the composition objects and forcing them to utilise the caching(getDistance) and partitioning(getPointsInArea) techniques. The class isolates the raw data from other classes to prevent data corruption and ensure the properties of the Travelling Salesman Problem(addPointInRoute & swapRoutePoints) are maintained. The tour constructor uses the addPointInRoute() function, and tour optimisers uses swapRoutePoints(), as seen from Figure1.

<u>Function</u>	<u>Argument</u>	<u>Return</u>	<u>Description</u>
TSP_Data()	Random Seed	Void	Constructor: Generate a random problem
getDistance()	Point & Point	Double	Checks the cache table or generates the Euclidean distance between two points
getPointInArea()	Area	List of Points	Uses the partitioning algorithm to fetch all the points within a geographical area
getRoute()	Null	List of Points	Returns current route
addPointInRoute()	Point & Position	Null	Adds point to the position in the route
swapRoutePoints()	Point & Point	Null	Swaps two points in the route

Table 4: Function Definition

7.3 TSP Modules Interfaces

Interfaces or pure virtual classes in C++(Section6.3.1) dictate the interaction through which to communicate with a block. The interface acts as a conduit between TSP_Data and the underlying algorithms and functionality(Figure43). Because the modules only need to interact statically with TSP_Data and they are situated in composition, a reference is all that is required for modules to access the API(Section7.2).

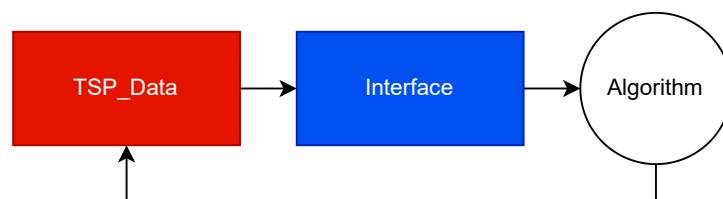


Figure 43: Modules Interface Interaction

The program employs this technique in four locations: the tour constructor, the tour optimiser, partitioning, and caching. For tour constructors and optimisers, the interface handles the uniform construction and destruction of every algorithm through a base process, runConstructor() and runOptimiser(). The partitioning interface requires the underlying partition type(Section3.5.2) to have both a construction method and a method for determining points within an area. Caching similarly requires a construction method and a distance between points. Interfaces

are a requirement because each implementation utilises a distinct memory structure and logic to achieve the same functionality, as shown in Figure45.

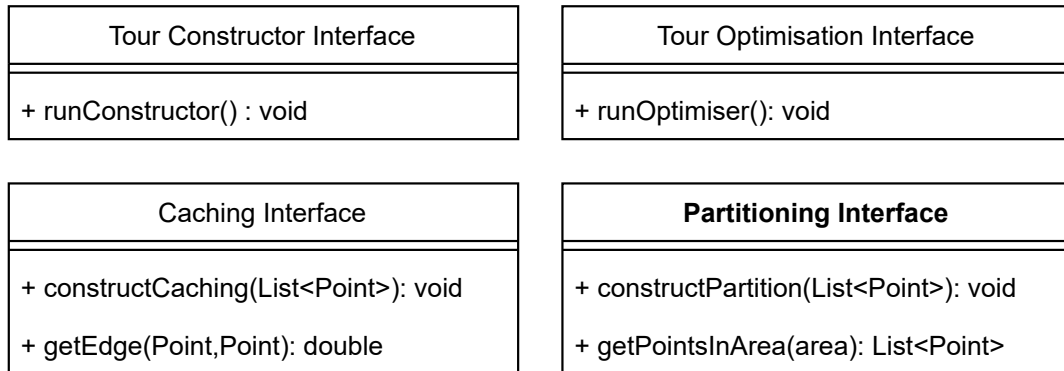


Figure 44: Interface Descriptions

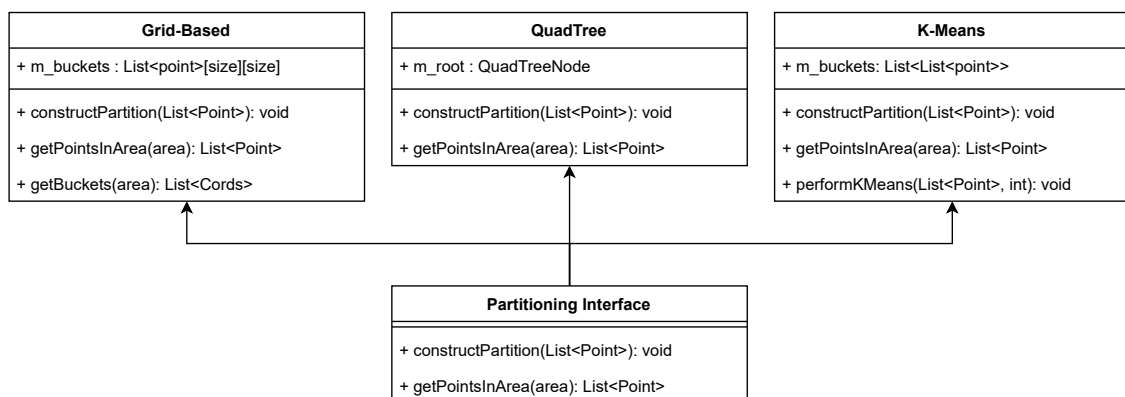


Figure 45: Partitioning Interface Example

7.4 TSP Projects

Projects are individual executables that compile shared and discrete code independently, each with its own set of compiler options and libraries. Mixing GUI applications and background applications presents numerous issues and increases complexity when the two applications are disjointed. Therefore, keeping each application's specific code and compiler options separate improves development time and user application efficiency. If the applications require shared code, that shared code should be placed in a static library(Section6.3), which is what the TSP Library is. This report requires four projects: Empirical Testing Suite, GUI output, Unit test, and TSP File Reader.

7.4.1 Empirical Testing Suite

The Empirical Testing Suite is a Command-Line Interface(CLI) application for selecting and starting the process of testing various algorithms, serving as the primary component of the report and following the methodology listed in Section5. It supports simultaneous testing and multiple test suite designs. It is responsible for creating, appending, and saving the result data to a file(More in Section7.4.4). There are multiple different test suites that serve different purposes:

<u>Name</u>	<u>Function</u>	<u>Aim</u>
Basic Test	Runs every algorithm for a given problem size	Generates simple data for each algorithm
Multi Test	Runs every algorithm for multiple different problem sizes	Generates a wide range of data for every algorithm
Dynamic Test	Runs only SLACIH and DLACIH for a given problem size	Generates data for the optimal depth of SLACIH and comparative performance of DLACIH
Dynamic Args Gradient Descent	Performs gradient descent on DLACIH arguments	Hyper-optimize the DLACIH arguments values
TSPLib Test	Runs DLACIH on every Euclidean TSP in TSPLib	Benchmark DLACIH against exact solutions

Table 5: Test Suites

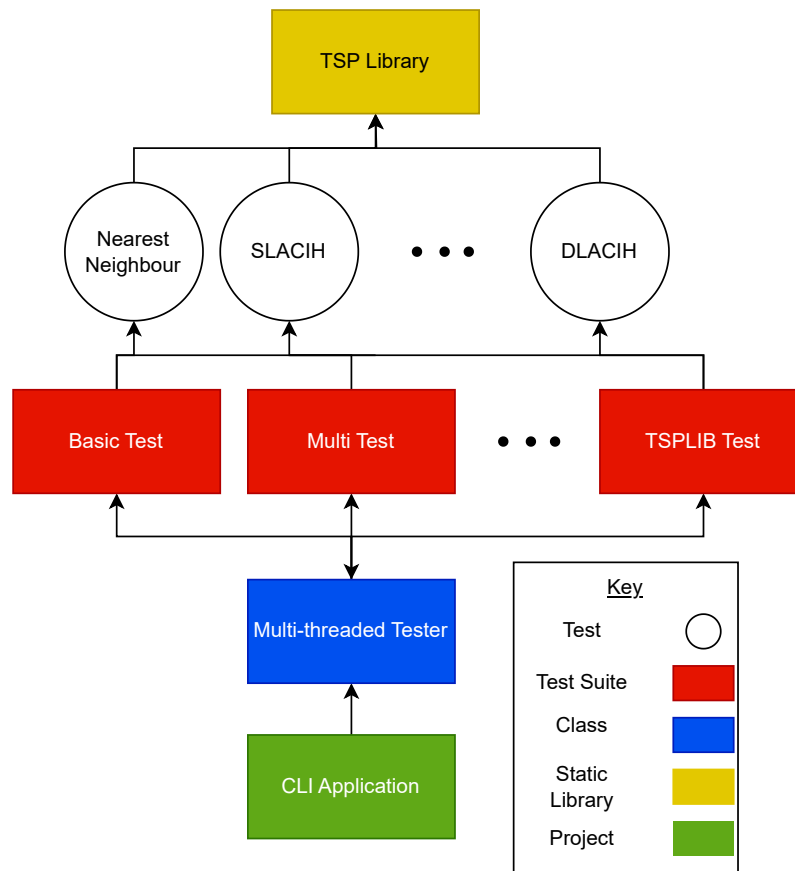


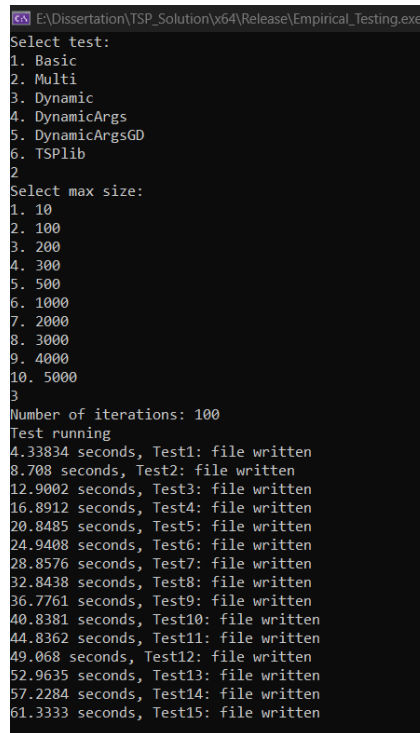
Figure 46: Empirical Testing Suite Design

The application block serves as the entry point, featuring only CLI menu logic and basic sanitation checks. It passes on the user inputs to the multi-threaded tester for execution. The menu collects inputs such as:

- Which test suite to run
- How many iterations to run
- How long it should run for
- What file type should it produce

The multi-threaded tester (Tester for short) is the core block, as it allows other classes to add tests to a queue for them to be run in parallel and adheres to the parameters set by the user. The Tester features an iterative, asynchronous loop that manages the continuous execution

of the selected test suite and periodically writes the results to a file(Section 7.4.4 & Figure 47). The Tester limits the number of active tests to the number of cores available and executes a new test only once a thread from the active thread pool becomes available in a FIFO queue. Equipped in the terminal is a CTRL+C break command to gracefully stop the test mid-run, preventing file corruption.



```

E:\Dissertation\TSP_Solution\64\Release\Empirical_Testing.exe
Select test:
1. Basic
2. Multi
3. Dynamic
4. DynamicArgs
5. DynamicArgsGD
6. TSPLib
2
Select max size:
1. 10
2. 100
3. 200
4. 300
5. 500
6. 1000
7. 2000
8. 3000
9. 4000
10. 5000
3
Number of iterations: 100
Test running
4.33834 seconds, Test1: file written
8.708 seconds, Test2: file written
12.9002 seconds, Test3: file written
16.8912 seconds, Test4: file written
20.8485 seconds, Test5: file written
24.9408 seconds, Test6: file written
28.8576 seconds, Test7: file written
32.8438 seconds, Test8: file written
36.7761 seconds, Test9: file written
40.8381 seconds, Test10: file written
44.8362 seconds, Test11: file written
49.068 seconds, Test12: file written
52.9635 seconds, Test13: file written
57.2284 seconds, Test14: file written
61.3333 seconds, Test15: file written

```

Figure 47: Empirical Test Suite

The test blocks(black circle in Figure 46) are individual tests that act as a wrapper(also known as an adaptor) for running an algorithm on the TSP Library with some given parameters. The test suite blocks(red rectangles in Figure 46) are predefined sets of tests that load the test blocks into the multi-threaded tester. Each test suite has a design it is trying to evaluate, and they do not behave equally. The test suite TSPLIB does not use random problems and instead solves the Euclidean TSP contained in TSPLIB from the smallest to the largest problem. The test suite Dynamic Args Gradient Descent(Table 5) uses the same five problems for each iteration of the gradient descent.

7.4.2 TSP GUI

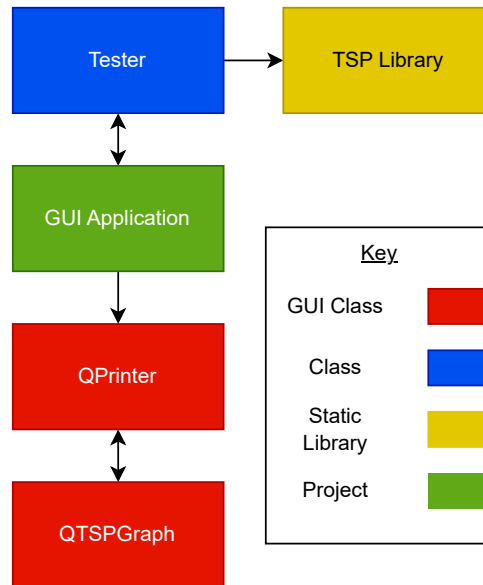


Figure 48: TSP GUI Design

Euclidean TSP belongs to the geometric space, therefore, the points and edges of the graph problem can be represented on a two-dimensional plane. This can be displayed on a GUI application to verify and analyse solutions. This is achieved through QT6 and a paint system for drawing diagrams(Figure49). The GUI application runs in two steps: it runs algorithms on a TSP, then displays the solution. The two steps do not interact, apart from passing the backend output into the frontend(Figure48).

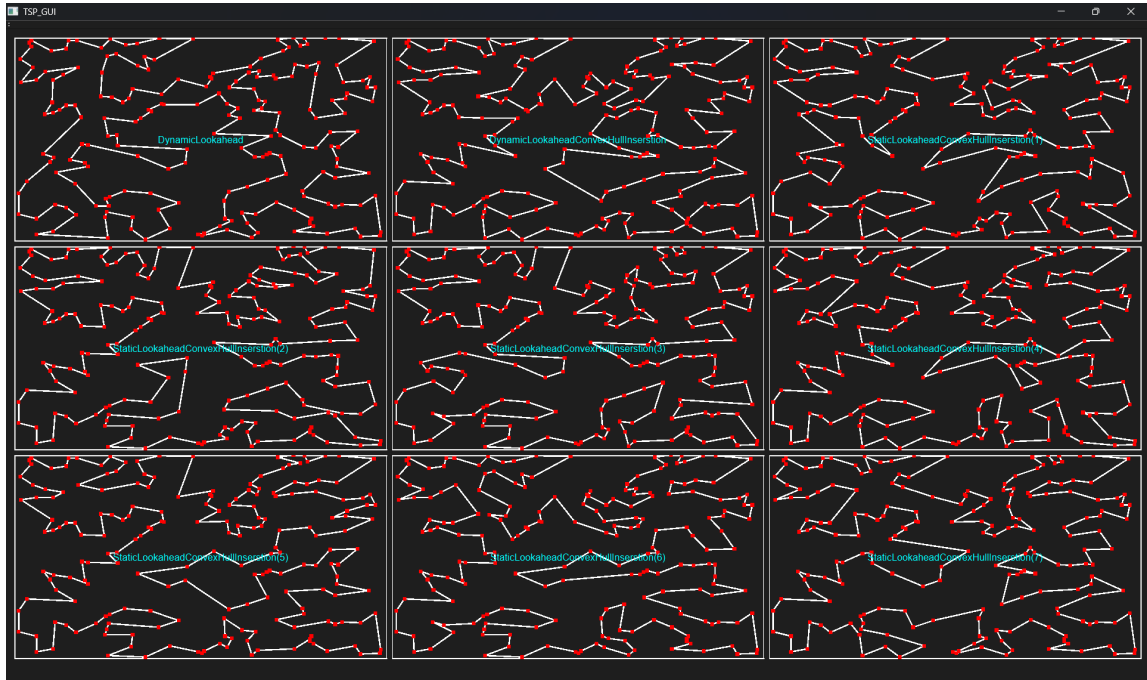


Figure 49: TSP GUI

7.4.3 Unit Test

C++ has a plethora of unit testing libraries, all with their pros and cons, but there are no wrong choices. GoogleTest^[124] is used for its simple integration into Visual Studio. In Figure39, the unit test encapsulates the entire code base as it can access and create any object needed for the test. This project followed the “art of unit testing”^[125] design for a good unit test, reducing complexity by testing only one variable and ensuring repeatable and predictable results.

```
[ RUN      ] SquareTest.Contains
[ OK       ] SquareTest.Contains (0 ms)
[ RUN      ] SquareTest.Overlap
[ OK       ] SquareTest.Overlap (0 ms)
[-----] 3 tests from SquareTest (1 ms total)

[-----] 1 test from StaticLookahead
[ RUN      ] StaticLookahead.Similarity
[ OK       ] StaticLookahead.Similarity (2 ms)
[-----] 1 test from StaticLookahead (2 ms total)

[-----] Global test environment tear-down
[=====] 31 tests from 6 test cases ran. (30 ms total)
[ PASSED  ] 31 tests.
```

Figure 50: Unit Test Completion

7.4.4 TSP File System

Tsp_file(Section7.1.1) refers to the naming convention and type of data that is being stored. For the use-case of storing test results, the file needs to be stored in a dynamically sized list of tsp_file entries for appending and reading the results. No standard exists, so the choice of file format is made by the developer. Text-based formats like JSON^[126] are human readable and scheme free. C++ does not have the modern compiler reflection feature^[127], which would allow the compiler to modify its own behaviour based on its own code. This feature is used by most JSON libraries outside C++ to serialise and deserialise class structure automatically; for C++, the conversion has to be manually defined which could introduce errors if not adequately tested.

The files produced by the empirical testing are illegible for human use due to excessive data in an illogical format. Therefore, an application, TSP_Reader, that can deserialise, analyse, and reproduce more legible data is beneficial. TSP_Reader uses the TSP file system to process the file, uses data processing to transform the data into a more readable form, and then uses the libxlsxwriter library(Section6.3.4) to create an Excel file with tables and graphs. The data is processed based on problem size(Figure51) and algorithm(Figure52).

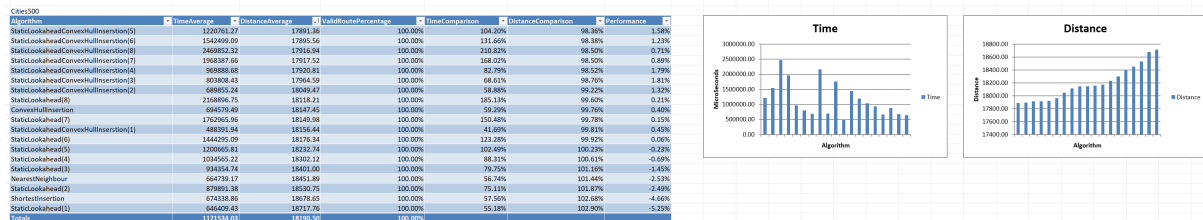


Figure 51: Excel File - Based On Size

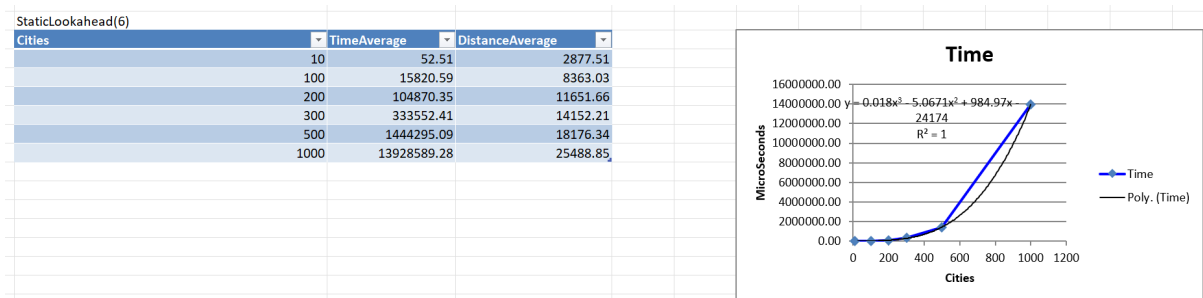


Figure 52: Excel File - Based On Algorithm

8 Implementation

The code for this project was source-controlled using [GitHub](https://github.com/AndrewF001/Dissertation), and the repository can be found at <https://github.com/AndrewF001/Dissertation>. The concepts mentioned in the system requirement(Section6.3), can be applied to the design of the system(Section7), and were implemented multiple times throughout the project. A few principal elements from the code have been selected to highlight the concepts used and key features. Nevertheless, due to the complexity and large volume of code, not every advanced technique could be demonstrated.

8.1 Polymorphism

For the Testing Suites class(Figure46), polymorphism ensures common functionality between test suites. Essential data is encapsulated between the classes, and pure virtual functions are defined to allow child classes to have individual functionality. Listing3 is a stripped-down version of the code in [test_class.h](#). *TestClass* is the pure virtual abstract parent class, and *RunTests()* is the interface code for the GUI application. The interface code is split into public(*RunTests()*) and private(*_test()*) functions, as the interface relies upon threads(*std::packaged_task*). Class *BasicTest* is the child class of *TestClass* and overrides the pure virtual function *Test()* with the test suite definition(Table5).

Listing 3: Testing Suites Polymorphic Classes

```
1 class TestClass
2 {
3 public:
4
5     std::shared_ptr<TSPFile> RunTests(size_t num_of_ittr = SIZE_MAX,
6         ↪ std::chrono::seconds timeout = std::chrono::seconds::max()) {
7         // package task to cancel if it takes too long
8         std::packaged_task<void()> task(std::bind(&TestClass::_test, this,
9             ↪ num_of_ittr));
10
11         // Wait for the task to finish or timeout
12         if (future.wait_for(timeout) != std::future_status::timeout) {
13             // Task was successfully completed
14             thr.join();
15             future.get();
16         }
```

```

15         return m_file;
16     }
17
18 private:
19     void _test(size_t num_of_ittr) {
20         size_t i = 0;
21         while (i < num_of_ittr) {
22             Test(); // Uses Virtual Function
23             i++;
24         }
25     }
26
27     virtual void Test() = 0; // Virtual Function
28 };
29
30 class BasicTest : public TestClass {
31 public:
32     void Test() override { // Override Function
33         // Control
34         auto seed = Tester::seed_gen();
35         auto cities = Tester::generateCities<Size>(seed);
36         TSPArgs args;
37
38         m_tester.TestAlgorithm<Size, ConstructionType::ConvexHullInsertion,
39             ↳ Opt>(seed, cities, args);
40         m_tester.TestAlgorithm<Size, ConstructionType::NearestNeighbour,
41             ↳ Opt>(seed, cities, args);
42         m_tester.TestAlgorithm<Size, ConstructionType::ShortestInsertion
43             ↳ ,Opt>(seed, cities, args);
44         m_tester.TestAlgmorithm<Size, ConstructionType::DynamicLookahead,
45             ↳ Opt>(seed, cities, args);
46         m_tester.TestAlgorithm<Size,
47             ↳ ConstructionType::DynamicLookaheadConvexHullInserstion,
48             ↳ Opt>(seed, cities, args);
49
50         for (size_t i = 1; i < 9; i++) {
51             args.max_depth = i;

```

```

46         m_tester.TestAlgorithm<Size, ConstructionType::StaticLookahead,
           ↪ Opt>(seed, cities, args);
47         m_tester.TestAlgorithm<Size,
           ↪ ConstructionType::StaticLookaheadConvexHullInserstion,
           ↪ Opt>(seed, cities, args);
48     }
49 };
50 };

```

8.2 Template Metaprogramming

Template Metaprogramming(TMP)(Section6.3.2) is a valuable tool for performance optimisation and compile-time polymorphism. One of the optimisations used in the program is to turn builder classes into monolithic classes if the type is known at compile time; this is how TSP_Library is implemented and can be found in [tsp_template.h](#). Using Enums and *constexpr* If statements, the choice selection of a child class can be decided at compile-time with compile-time polymorphism. Listing4, an edited snippet of code from [tsp_template.h](#); looks at how this compile-time polymorphism is applied to *constructTour()* with the use of *ConstructionType* Enum and *Construction* template variable. By assigning the template variable to the Enum values(Line12) and modifying the builder function(*constructTour()*, Line24) to the creation of predefined children classes, the use of a V-Table is avoided. As templates are instantiated for each template variable(Section6.3.2), there are 96 different versions of the *TspTemplate* being compiled, but adding more algorithms increases the number rapidly:

$$\# \text{ of template instantiation} = C \cdot O \cdot CT \cdot PT \cdot T$$

$$96 = 8 \cdot 2 \cdot 3 \cdot 2 \cdot 1$$

Where: *C* is the number of *ConstructionType*, *O* is the number of *OptimisationType*, *CT* is the number of *CachingType*, *PT* is the number of *PartitioningType*, *T* is the number of *TSPTType*.

Listing 4: TSP Library Template Meta-Programming

```

1  enum class ConstructionType {
2      DynamicLookahead,
3      DynamicLookaheadConvexHullInserstion,

```



```

4     StaticLookaheadConvexHullInserstion,
5     StaticLookahead,
6     NearestNeighbour,
7     ShortestInsertion,
8     ConvexHullInsertion,
9     ConvexHull,
10 };
11
12 template <class TSPTYPE, size_t Size, CachingType Caching, PartitioningType
    ↳ Partitioning, ConstructionType Construction, OptimisationType
    ↳ Optimisation>
13 class TspTemplate {
14 public:
15     void run(const TSPArgs& args) {
16         m_data.initalisePartition();
17         m_data.initaliseCache();
18         constructTour(args);
19         optimiseTour(args);
20         finaliseOutput();
21     };
22
23 private:
24     void constructTour(const TSPArgs& args) {
25         if constexpr (Construction ==
    ↳ ConstructionType::StaticLookaheadConvexHullInserstion)
26             StaticLookaheadConvexHull<TSPTYPE, Size, Caching,
    ↳ Partitioning>(args.max_depth).constructTour(m_data);
27
28         ... Multiple If Statements for all the values of ConstructionType ...
29
30         if constexpr (Construction ==
    ↳ ConstructionType::DynamicLookaheadConvexHullInserstion)
31             DynamicLookaheadConvexHull<TSPTYPE, Size, Caching,
    ↳ Partitioning>(args.max_depth,
    ↳ args.dynamic_args).constructTour(m_data);
32     };
33 };

```

8.3 API

APIs are used everywhere but are mostly used in-depth by `TSP_Data`. It is the central block that contains the Travelling Salesman Problem data and abides by its constraints. It also manages the interoperability of all the blocks and ensures software engineering optimisation is enforced. `TSP_Data` has many public functions (Section 7.2 & Figure 42), but Listing 5 looks at how caching (Section 3.5.1) is managed for fetching the distance of an edge. Obscuring the caching mechanism behind an API call also requires that `m_cities` (Line 24) cannot be accessed from outside the class, potentially letting other classes bypass the caching mechanism.

Listing 5: TSP Data Interface

```
1 class TspDataTemplate {
2 public:
3     double getDistance(cityID city1, cityID city2) {
4         if constexpr (Caching == CachingType::full)
5             return m_cache[city1][city2];
6
7         double result;
8         if constexpr (Caching == CachingType::partial) {
9             result = m_cache[city1][city2];
10            // Check if the value is already in the cache
11            if (result != DBL_MAX)
12                return result;
13        }
14
15        // Calculate the distance and store result
16        result = m_cities[city1].getDistance(m_cities[city2]);
17        if constexpr (Caching == CachingType::partial)
18            m_cache[city1][city2] = result;
19
20        return result;
21    };
22
23 private:
24     std::array<TSPType, Size> m_cities; // Array of cities
25 };
```

8.4 Static Look-Ahead Cheapest Insertion Heuristic

The development and implementation of SLACIH(Section4.2) is the project's main focus, and its implementation serves as the proof of work. As described above, it builds upon polymorphism, templates, and API calls to remain adaptable and optimised. It incorporates all the optimisations used by the other algorithms but also includes several unique algorithmic optimisation techniques:

- Early exit if the current distance exceeds the best-known solution(Line88)
- Use of closeness proximity(Section3.5.2) by only considering points within the distance of the current best solution(Line62)
- Early exit if no points are within range of a partial_route(Line83).

Listing6 shows the complete, mostly unedited, C++ code of the algorithm, which can also be found in [static_lookahead.h](#).

Listing 6: Static Look-Ahead Cheapest Insertion Heuristic

```
1 template<class TSPTType, size_t Size, CachingType Caching, PartitioningType Partitioning>
2 class StaticLookahead : public ConstructionBase<TSPTType, Size, Caching, Partitioning>
3 public:
4 void _constructTour(TspDataTemplate& data) override {
5     data.setCityPos(0, 0); // Randomly select the first city
6     lookaheadInsertion(data, m_depth);
7 };
8
9 static void lookaheadInsertion(TspDataTemplate& data, size_t depth) {
10     const size_t additions = Size - data.getRouteSize();
11     for (size_t i = 0; i < additions; i++) {
12         auto [closest_point, route_position] = FindClosestPoints(data, depth);
13         data.setCityPos(closest_point, route_position + 1);
```

```

14     }
15 }
16
17 static std::pair<cityID, size_t> FindClosestPoints(TspDataTemplate& data, size_t depth) {
18     double min_dist = DBL_MAX;
19     std::pair<cityID, size_t> output;
20
21     // Check if depth is greater than the number of cities left
22     if (Size - data.getRouteSize() < depth)
23         depth = Size - data.getRouteSize();
24     // Check each city
25     for (cityID idx = 0; idx < Size; idx++) {
26         if (data.isCityInRoute(idx))
27             continue; // If in route continue
28
29         // Find the shortest partial route for the city
30         auto [distance, position] = ShortestRoute(data, idx, depth, min_dist);
31         if (distance < min_dist) {
32             min_dist = distance;
33             output = { idx, position };
34         }
35     }
36     return output;
37 }
38 private:
39 const size_t m_depth;
40 static std::pair<double, cityID> ShortestRoute(TspDataTemplate& data, cityID point, size_t depth, const double

```

```

    ↪ best_distance) {
41  std::pair<double, cityID> output = { DBL_MAX, SIZE_MAX }; // distance, position
42  std::vector<cityID> partail_route(3);
43  partail_route[1] = point;
44
45  // find the best place to insert the point
46  const auto& route = data.getRoute();
47  for (size_t i = 0; i < data.getRouteSize(); i++) {
48      double distance = data.calcDeivation(partail_route[1], data.getRouteCityID(route[i]),
    ↪ data.getRouteCityID(route[i + 1]));
49      if (distance < output.first) {
50          output.first = distance;
51          output.second = i;
52      }
53  }
54  // Add it to partail route
55  partail_route[0] = data.getRouteCityID(route[output.second]);
56  partail_route[2] = data.getRouteCityID(route[output.second + 1]);
57  // Repeat for lookaheads
58  // Find closest point that is a part of partial route
59  double offset = best_distance * best_distance;
60  std::vector<cityID> city_search = data.getCitiesInArea(partail_route[1], offset);
61
62  for (size_t i = 0; i < depth - 1; i++) {
63      double min_dist = DBL_MAX;
64      std::pair<cityID, size_t> add_point = { SIZE_MAX, SIZE_MAX };
65      for (const auto& city : city_search) {

```

```

66     if (data.isCityInRoute(city))
67         continue;
68     // Check if point is already in partail route
69     if (std::any_of(partail_route.begin() + 1, partail_route.end() - 1, [city](const auto& route_point)
70         ↪ { return route_point == city; }))
71         continue;
72
73     for (size_t j = 0; j < partail_route.size() - 1; j++) {
74         double distance = data.calcDeivation(city, partail_route[j], partail_route[j + 1]);
75         if (distance < min_dist) {
76             min_dist = distance;
77             add_point = { city, j };
78         }
79     }
80     if (add_point.first == SIZE_MAX) // No point found
81         break;
82     // add closest point to partail route
83     partail_route.insert(partail_route.begin() + add_point.second + 1, add_point.first);
84     output.first += min_dist;
85     if (output.first > best_distance)
86         break;
87 }
88 return output;
89 }
90 };

```

8.5 Dynamic Look-Ahead Cheapest Insertion Heuristic

DLACIH expands on SLACIH with a optimal depth function(Listing7), afterwards it is unchanged from the base algorithm. The cost function is defined in Equation6, which can be seen in Function *optimalDepth()*(line7) with the arguments being determined by the struct *DynamicArgs*.

Listing 7: Dynamic Look-Ahead Cheapest Insertion Heuristic Depth Function

```
1 struct DynamicArgs { // Numbers are from Gradient Descent analysis
2     double multiplier = 1.2348586676954691;
3     double logarithm = 0.90649189814814557;
4     double constant = -1.6065204475308614;
5 };
6
7 static size_t optimalDepth(size_t size, size_t max_depth, DynamicArgs& args) {
8     size_t optimal = (size_t)std::round(args.multiplier *
9         ↪ std::log(args.logarithm * size) - args.constant);
10
11     if (optimal == 0)
12         optimal = 1;
13
14     return std::min(optimal, max_depth);
15 }
```

8.6 Gradient Descent

Gradient descent is a useful tool for optimising the depth function arguments with empirical data as the cost function is convex. To obtain better results that are not hyper-optimised, taking the summation of a set of problems is beneficial, as displayed by Listing8 where the *NumOfProblems* = 5(Line1). How the gradient descent is set up can be seen in [dynamic_args_gradient_descent.h](#).

Listing 8: Graident Descent Cost Function

```
1 NumOfProblems = 5;
2 double cost_function(const gsl_vector* v, void* params) {
3     double score = 0;
4     std::array<std::unique_ptr<std::array<Type2d, Size>>, NUMOFPROBLEMS>*
```

```

5         ↪ m_cities = (std::array<std::unique_ptr<std::array<Type2d, Size>>,
6         ↪ NUMOFPROBLEMS>*)params;

7
8     DynamicArgs d_args = { .multiplier = gsl_vector_get(v, 0), .logrithm =
9         ↪ gsl_vector_get(v, 1), .constant = gsl_vector_get(v, 2) };

10
11     TSPArgs args{ .max_depth = 10, .timeout_ms =
12         ↪ std::chrono::milliseconds(300000), .dynamic_args = d_args};

13
14     for (size_t i = 0; i < NUMOFPROBLEMS; i++) {
15         auto algorithm = std::make_unique<TspTemplate<Type2d, Size,
16             ↪ CachingType::full, PartitioningType::quadTree,
17             ↪ ConstructionType::DynamicLookaheadConvexHullInserstion,
18             ↪ OptimisationType::TwoOpt>>(AREA, ((*m_cities)[i]),
19             ↪ GenerationType::rectangle, 0);
20         score += algorithm->run(args).final_distance;
21     }
22     return score;
23 }

```

9 Project Management

Before the official project start date, a feasibility study was conducted, including the creation of a simple prototype over the 2024 summer break to validate the algorithm. In doing so, it highlighted opportunities for future work and areas of improvement. The work conducted could not be transferred to the main project, as an entire redesign of the system was necessary with the addition of TSP_Data and the project's planned features.

At the start of the project, a proposed timeline defining the development of major features was produced (Figure 53). Key milestones were identified that needed to be reached before moving on, as each system was built on top of the other. This led to the natural conclusion of using the waterfall project management model^[128], requiring each step to design, implement, and test the current feature. This process ensured that backtracking or refactoring was not needed as the trust in existing code for future development was guaranteed.

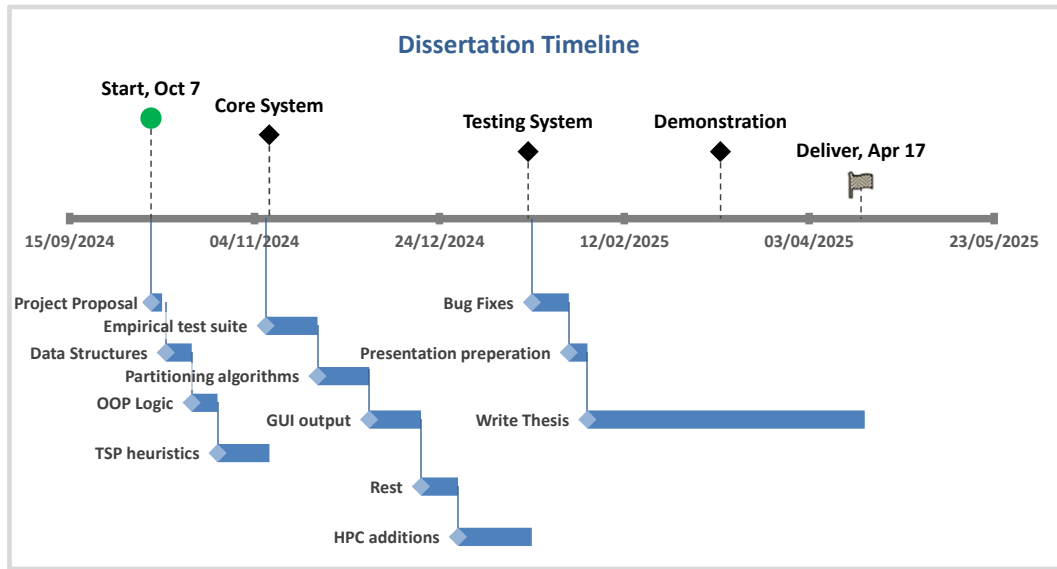


Figure 53: Project Timeline

The major milestones split the project between the core system(TSP Library) and application-specific development. Both of the major milestones had minor milestones that structured the development process of each system(Figure53). Similar to many projects, the project overran due to ambitious goals, leading to the high-performance computing feature being removed from the project scope in order to meet the final delivery date. Nonetheless, the quality of work and development efficiency achieved throughout the project were the highest delivered.

10 Unit Testing

Unit Testing is part of the waterfall development cycle(Section9) to ensure current components function as intended. The tests should follow the design principles(Section7.4.3) of keeping it simple with one independent variable for testing. GoogleTest^[124] was used as the testing framework, but it functions the same as most testing frameworks. The tests should focus on the smallest components possible to provide the greatest coverage across the system. The testing uses a result-driven approach by using *asserts()* on the value produced against the expected value, providing long-standing coverage if the code unexpectedly changes behaviour.

10.1 Basic Types

The basic types such as point, edge, and square are used throughout the program and therefore should be robust. They are tested on creation and on each of their functions with boundary testing. Listing9 shows the unit testing of *Point2D* class constructor and *gradient()* function against predetermined values calculated manually.

Listing 9: Point Unit Test

```
1 TEST(PointTest, Creation) {
2     Point2D p1{ 0, 0 };
3     EXPECT_EQ(p1.m_x, 0);
4     EXPECT_EQ(p1.m_y, 0);
5
6     Point2D p2{ 10, 10 };
7     EXPECT_EQ(p2.m_x, 10);
8     EXPECT_EQ(p2.m_y, 10);
9 }
10
11 TEST(PointTest, Gradient) {
12     Point2D p1{ 0, 0 };
13     Point2D p2{ 10, 10 };
14     Point2D p3{ 10, 0 };
15     Point2D p4{ 0, 10 };
16     Point2D p5{ -10, -10 };
17
18     EXPECT_EQ(p1.gradient(p2), 1); // Positive slope
19     EXPECT_EQ(p1.gradient(p5), 1); // Same slop reverse position
20     EXPECT_EQ(p1.gradient(p3), 0); // Horizontal line
21     EXPECT_EQ(p1.gradient(p4), INFINITY); // Virtical line
22 }
```

10.2 Quad Tree

The partitioning algorithms have more sophisticated functionality, with more complex data structures, which rely on the basic data types to function. Exhaustive boundary testing cannot be applied because there is a non-finite number of Quad Tree structures, but testing the sub-procedure can guarantee each step. Listing10 shows the creation, insertion, and

searching of a basic quad-tree to test its subdividing properties.

Listing 10: Quad tree Unit Test

```
1 TEST(QuadTreeTest, BasicSearch) {
2     QuadTree tree(Square{ {0, 0}, 100, 100 });
3     tree.insert(0, Point2D{ 60, 60 }); // Root
4     tree.insert(1, Point2D{ 30, 30 }); // Top left
5     tree.insert(2, Point2D{ 70, 70 }); // Bottom right
6     tree.insert(3, Point2D{ 30, 70 }); // Bottom left
7     tree.insert(4, Point2D{ 70, 30 }); // Top right
8
9     auto found = tree.contains(Square{ {0, 0}, 40, 40 });
10    check_result(found, { 1 });
11 }
```

10.3 Look-Ahead Algorithm

Testing a look-ahead algorithm is infeasible as it requires doing a dry run of the algorithm manually. There are also too many parameters to cover all test cases exhaustively. Therefore, applying Stability Tests to ensure the code remains the same between versions and does not crash is acceptable. Listing11 confirms that the class compiles and runs without crashing for example data.

Listing 11: SLACIH Stability Test

```
1 TEST(StaticLookahead, Similarity) {
2     Square area = { {0, 0}, 1000, 1000 };
3     std::mt19937 engine(1);
4
5     auto cities = TspDataTemplate<Type2d, 50, CachingType::full,
6         ↳ PartitioningType::quadTree>::generateCities(area,
7         ↳ GenerationType::rectangle, engine);
8
9     auto lookahead = std::make_unique<TspTemplate<Type2d, 50, CachingType::full,
10        ↳ PartitioningType::linearSearch,
11        ↳ ConstructionType::StaticLookaheadConvexHullInserstion,
12        ↳ OptimisationType::None>>(area, cities)->run({1});
13
14     auto linear = std::make_unique<TspTemplate<Type2d, 50, CachingType::full,
```

```

    ↪ PartitioningType::linearSearch,
    ↪ ConstructionType::ConvexHullInsertion, OptimisationType::None>>(area,
    ↪ cities)->run({});
9
10 EXPECT_EQ(lookahead.route, linear.route);
11 }

```

10.4 JSON Conversion

Testing on the JSON conversion code helps protect against file corruption. As the conversion is symmetric, round case testing can be used to compare the written and read data. Listing12 demonstrates the symmetrically properties by comparing the original object with the round case object to determine they are identical.

Listing 12: Json Round Case Test

```

1 TEST(JsonConversionTest, RunModeTest) {
2     RunMode s; // Original value
3     s.num_cities = 10;
4     s.genType = GenerationType::rectangle;
5     s.Caching = CachingType::full;
6     s.Partitioning = PartitioningType::quadTree;
7     s.Construction = ConstructionType::StaticLookaheadConvexHullInsertion;
8     s.Optimisation = OptimisationType::TwoOpt;
9     s.args = { 1, 2, std::chrono::milliseconds(100) };
10
11     rapidjson::Document doc;
12     doc.SetObject();
13     rapidjson::Value val;
14     jsonconversion::runModeToJson(s, val, doc.GetAllocator()); // Write JSON
15     doc.AddMember("runMode", val, doc.GetAllocator());
16     RunMode r;
17     jsonconversion::JsonToRunMode(r, doc["runMode"]); // Read JSON
18
19     EXPECT_EQ(s.num_cities, r.num_cities); // Round case testing
20     EXPECT_EQ(s.genType, r.genType);
21     EXPECT_EQ(s.Caching, r.Caching);
22     EXPECT_EQ(s.Partitioning, r.Partitioning);

```

```
23 EXPECT_EQ(s.Construction, r.Construction);
24 EXPECT_EQ(s.Optimisation, r.Optimisation);
25 EXPECT_EQ(s.args, r.args);
26 }
```

11 Evaluation

The project can be evaluated on its software engineering achievement(Section6) and the proposed SLACIH and DLACIH algorithm performance(Section5); each has its challenges, with clear goals and strategies outlined. The evaluation in this section has been backed up with evidence of data in achieving the goals.

11.1 Functional & Non-Functional Requirements

For the software engineering achievements, precise requirements were set in the functional and non-functional requirements(Section6.1). The requirements in Table2&3 have been evaluated with what has been achieved in the Tables6&7:

<u>Requirement</u>	<u>Achieved</u>
1.a) Graph Theory	Two-dimensional graphs are represented in 2d.h and tested
1.b) TSP Constraints	tsp_data_template.h correctly enforces the constraints of the Travelling Salesman Problem
1.c) Euclidean TSP	The <i>getDistance()</i> function correctly calculates the Euclidean distance
2.a) Facilities for Algorithms	tsp_template.h allows multiple algorithms to be used simply by changing a variable value
2.b) Benchmarking Algorithm	timer.h is a custom-built high-resolution clock timer specifically built for this project, and extended tsp_file to include all the information needed
2.c) Algorithm Fairness	The use of compile-time polymorphism treats every algorithm as individual sets of code, each being inlined and optimised by the compiler without knowledge of the other algorithm, and each algorithm worked with the same API calls and features
2.d) Testing Suite	There are 5 test suites (Table 5), and developed on a stable testing platform
3.a) Display TSP Solutions	There is a GUI application for displaying one or many TSP solutions
4.a) Collect Test Data	TSP files with test data are created in JSON format
4.b) Analysis Test Data	Excel tables and graphs and programmatically produced from the TSP files
5.a) Unit Test	The unit tests covers TSP_Library but not all aspects all aspects of the code base

Table 6: Functional Requirements Achieved

<u>Requirement</u>	<u>Achieved</u>
The user should be able to select the type of algorithm they want easily	The user can efficiently run pre-made algorithm setups, however, because compile-time polymorphism requires knowledge of which algorithm will be run beforehand, unless the user is a programmer, they are unlikely able to configure the bespoke algorithm
The program should never fail or crash	The testing suites were run for multiple days without a crash or invalid TSP solution
The program should be responsive and utilise all computing power	The program does not hang and uses all available cores when running multiple solutions
The program should be adaptable and fill the needs of each executable	There is no repeated code to cover the same functionality
The GUI should be easy to read for all users e.g. inclusive of those with colour blindness	The colour palette does not clash with common colour blindness, but the graphics are small and lack graphical fidelity features like anti-aliasing

Table 7: Non-Functional Requirements Achieved

11.2 Empirical Testing Results

Following this report's research, methodology, design and testing; empirical testing is the lead evaluation for comparing the Static & Dynamic Look-Ahead Cheapest Insertion Heuristic against other known heuristics. Every algorithm(Table8) used Quad-Tree Partitioning, Full Caching, and tour optimiser with 2-opt swap, because from testing this was shown to be the best set-up for solution result and run time.

Acronym	Algorithm
DLACHCIH	Dynamic Look-Ahead Convex Hull Cheapest Insertion Heuristic
DLACIH	Dynamic Look-Ahead Cheapest Insertion Heuristic
SLACHCIH(X)	Static Look-Ahead Convex Hull Cheapest Insertion Heuristic, Depth = X
SLACIH(X)	Static Look-Ahead Cheapest Insertion Heuristic, Depth = X
NNS	Nearest Neighbour Search Heuristic
CHCIH	Convex Hull Cheapest Insertion Heuristic
CIH	Cheapest Insertion Heuristic

Table 8: Algorithm Acronym Key

Each test instance problem was randomly generated. and ran on each algorithm for six problem sizes, which were chosen to represent small, medium, and large problems. Multiple depths of the static look-ahead were evaluated to prove and develop the dynamic look-ahead optimal depth curvature. This report theorised lots of dynamic algorithms(Section4.4), but for this testing, only static dynamic depth optimisation(Section4.4.1) was considered. As DLACIH calls upon SLACIH code and therefore would generate the same solution, running the code twice was avoided to speed up testing. The distance values for DLACIH and DLACHCIH are based on the static depth it would have chosen to run SLACIH, and the times can be assumed to be identical, but left blank for clarity.

Each value in the Table9&10 is the mean average of 1,000 problems being solved. This results in 114,000 TSP problems being solved and 240,540,000 Nodes being routed. This required multiple weeks of constant computational running and tens of thousands of CPU hours.

Table 9
Empirical Testing Distance Results

Algorithm	Problem Size					
	10 Nodes	100 Nodes	200 Nodes	300 Nodes	500 Nodes	1000 Nodes
DLACHCIH	2864.35	8138.31	11385.62	13902.06	17895.56	25180.95
DLACIH	2898.15	8420.75	11685.38	14256.36	18176.34	25436.01
SLACHCIH(0)	2864.35	8191.25	11490.84	14062.30	18156.44	25651.97
SLACHCIH(1)	2864.57	8151.50	11433.55	13988.46	18049.47	25467.84
SLACHCIH(2)	2864.46	8140.39	11398.91	13939.97	17964.59	25327.55
SLACHCIH(3)	2864.55	8138.31	11396.14	13908.57	17920.81	25256.85
SLACHCIH(4)	2864.47	8143.77	11385.62	13900.00	17891.36	25210.99
SLACHCIH(5)	2864.45	8150.61	11394.39	13902.06	17895.56	25180.95
SLACHCIH(6)	2864.45	8162.80	11398.80	13906.92	17917.52	25177.56
SLACHCIH(7)	2864.45	8173.54	11418.06	13933.31	17916.94	25184.09
SLACIH(0)	2898.15	8620.55	11990.80	14581.91	18717.76	26242.28
SLACIH(1)	2886.81	8540.03	11885.12	14441.23	18530.75	25998.57
SLACIH(2)	2885.52	8480.63	11783.13	14343.36	18401.00	25802.91
SLACIH(3)	2878.55	8420.75	11721.57	14256.36	18302.12	25658.64
SLACIH(4)	2878.31	8373.10	11685.38	14199.79	18232.74	25545.87
SLACIH(5)	2877.51	8363.03	11651.66	14152.21	18176.34	25488.85
SLACIH(6)	2874.40	8343.51	11628.51	14132.63	18149.98	25436.01
SLACIH(7)	2873.75	8336.78	11605.18	14121.65	18118.21	25419.93
NNS	2888.89	8494.45	11828.17	14379.98	18451.89	25847.28
CHCIH	2864.35	8190.42	11492.43	14063.86	18147.45	25581.89
CIH	2898.15	8619.66	11991.10	14571.81	18678.65	26132.84

Source: : Average of 1000 random problems

Note: : Red values are best in class

Table 10
Empirical Testing Time(Microseconds) Results

Algorithm	Problem Size					
	10 Nodes	100 Nodes	200 Nodes	300 Nodes	500 Nodes	1000 Nodes
DLACHCIH	N/A	N/A	N/A	N/A	N/A	N/A
DLACIH	N/A	N/A	N/A	N/A	N/A	N/A
SLACHCIH(0)	28.83	5051.86	36190.85	111427.81	488391.94	3620307.70
SLACHCIH(1)	25.37	7261.49	48326.42	155657.94	689855.24	6353401.04
SLACHCIH(2)	24.09	8234.30	56582.62	182868.75	803808.43	7537333.35
SLACHCIH(3)	24.06	10040.87	69633.18	218068.36	969888.68	9223702.34
SLACHCIH(4)	24.41	12184.33	87280.23	272952.20	1220761.27	11632562.85
SLACHCIH(5)	24.44	14956.88	108331.81	343997.62	1542499.09	14895643.66
SLACHCIH(6)	24.71	17467.33	134173.32	432163.16	1968387.66	19003165.87
SLACHCIH(7)	24.18	20843.73	164704.99	533267.40	2469852.32	23702107.36
SLACIH(0)	39.98	7902.99	52819.91	155816.52	646409.43	4608439.04
SLACIH(1)	43.13	9994.76	65427.01	205581.55	879891.38	7931265.55
SLACIH(2)	42.97	10616.85	68674.72	216557.75	934354.74	8607775.94
SLACIH(3)	45.30	11922.03	76106.33	238098.60	1034565.22	9749928.92
SLACIH(4)	49.89	13914.21	88204.19	271463.36	1200665.81	11465799.93
SLACIH(5)	52.51	15820.59	104870.35	333552.41	1444295.09	13928589.28
SLACIH(6)	59.00	19406.64	127248.89	394223.06	1762965.96	17155433.92
SLACIH(7)	58.87	22896.07	154385.62	479555.42	2168896.75	21146002.81
NNS	51.85	8163.56	53758.76	160573.75	664739.17	4624162.49
CHCIH	74.79	16037.53	61741.36	173547.76	694579.49	4909795.79
CIH	39.75	7892.48	53696.50	158542.26	674338.86	4904754.48

Source: : Average of 1000 random problems

For every problem size(Table9), SLACHCIH provided the best solution from the algorithms tested, although the same depth was not used. As theorised, as the problem size increases, a higher look-ahead depth is optimal. This is where DLACHCIH's optimal depth function proved to be highly fit for purpose. However, the gradient descent variable optimisation failed to find the optimal curve that could otherwise be found through statistical analysis. The data also correctly demonstrated the theory that a higher depth is not always better. This is due to the pitfalls the look-ahead design introduces when it searches for nodes far away from a cluster(Section4.3).

From the conventional heuristic, CHCIH is the leading algorithm, but none could be better than DLACHCIH for any problem size. Using Equation10 to compare the solution results vs DLACHCIH in a Table11:

$$\frac{\text{Conventional}_{Result}}{\text{DLACHCIH}_{Result}} - 1 \quad (10)$$

Table 11: DLACHCIH vs Conventional Algorithm

Conventional Algorithm	Problem Size					
	10 Nodes	100 Nodes	200 Nodes	300 Nodes	500 Nodes	1000 Nodes
NNS	0.856%	4.376%	3.887%	3.438%	3.109%	2.646%
CHCIH	0.000%	0.640%	0.937%	1.164%	1.408%	1.592%
CIH	1.18%	5.914%	5.318%	4.818%	4.376%	3.780%

Note: : For any value above 0% DLACHCIH performs better

11.3 TSPLib Results

When testing the TSPLib exact solution, each problem had a unique size; therefore, DLACIH was used as it performs best for an extensive range of problems. This was the only algorithm tested, as trying to hyper-optimize the depth would not represent the proposed algorithm fairly, as this would introduce a new independent variable to the evaluation. Table12 outlines the distance against the exact solution for the 56 smallest Euclidean TSPLIB problems, which is all that could be computed in a reasonable time:

Problem	Exact Solution	DLACIH Solution	Difference From Exact(%)
eil51	426	442.918	3.971%
eil76	538	588.222	9.335%
pr76	108159	109872	1.584%
rat99	1211	1284.34	6.056%
rd100	7910	8028.18	1.494%
eil101	629	688.237	9.418%
pr107	44303	46065.5	3.978%
pr124	59030	59931.4	1.527%
bier127	118282	121483	2.706%
pr136	96772	98678.4	1.97%
pr144	58537	63973.6	9.287%
pr152	73682	75735.9	2.788%
u159	42080	45610.3	8.389%
rat195	2323	2536.77	9.202%
d198	15780	16714.3	5.921%
tsp225	3916	4127.81	5.409%
ts225	126643	141079	11.399%
pr226	80369	82470	2.614%
gil262	2378	2531.35	6.449%
pr264	49135	52329.2	6.501%
a280	2579	2729.76	5.846%
pr299	48191	51572.4	7.017%
rd400	15281	16233	6.23%
fl417	11861	12273	3.474%
pr439	107217	116485	8.644%
pcb442	50778	54220.4	6.779%
d493	35002	37947.2	8.414%
u574	36905	39236.6	6.318%
rat575	6773	7502.05	10.764%
p654	34643	35466.9	2.378%
d657	48912	52769.7	7.887%
u724	41910	45781.4	9.237%

rat783	8806	9638.47	9.453%
pr1002	259045	284101	9.672%
u1060	224094	242810	8.352%
vm1084	239297	261821	9.413%
pcb1173	56892	62368	9.625%
d1291	50801	57963.3	14.099%
rl1304	252948	271365	7.281%
rl1323	270199	291955	8.052%
nrl1379	56638	61127.3	7.926%
fl1400	20127	21485.2	6.748%
u1432	152970	165038	7.889%
fl1577	22249	24216.5	8.843%
d1655	62128	68199.4	9.772%
vm1748	336556	370251	10.012%
u1817	57201	64190.6	12.219%
rl1889	316536	344067	8.698%
d2103	80450	83940.4	4.339%
u2152	64253	72821.7	13.336%
u2319	234256	247967	5.853%
pr2392	378032	421713	11.555%
pcb3038	137694	150113	9.019%
fl3795	28772	30407	5.683%
Total	4642099	5013946.707	8.01%

Table 12: TSPLib Result

Using the Equation9 to take the average of all the results evenly, the average percentage difference is 7.2345% greater than the exact solution. This is a respectable result, but is insignificant to current benchmarking heuristics that are able to get within 0.5% of the exact solution^[129].

11.4 Time Analysis

Evaluating the time complexity of an algorithm based on run time, especially when variables affect the time complexity, can be difficult. The theory for the algorithm design puts the time complexity at $O(n^5)$ (Equation4). In reality, the time complexity is closer to n^3 for low

depths, but would be n^5 when $depth = n$. When dividing the time by the growth (Table13), n & n^2 continue to grow exponentially, n^4 & n^5 steadily decrease, while n^3 remains steady, suggesting the time complexity is $O(n^3)$ when the depth is seven. This is backed up when generating trend lines for a line graph of time vs size (Figure54), where n^3 is the dominant factor. While the timings for every algorithm are present in Table10, hesitation in comparing them is advised as the classical heuristics were not as meticulously optimised as SLACIH.

Cities	TimeAverage	t/n	t/n^2	t/n^3	t/n^4	t/n^5
10	24.18	2.4183	0.24183	0.024183	0.002418	0.000242
100	20843.73	208.43727	2.084373	0.020844	0.000208	2.08E-06
200	164704.99	823.52493	4.117625	0.020588	0.000103	5.15E-07
300	533267.40	1777.55799	5.925193	0.019751	6.58E-05	2.19E-07
500	2469852.32	4939.70464	9.879409	0.019759	3.95E-05	7.9E-08
1000	23702107.36	23702.10736	23.70211	0.023702	2.37E-05	2.37E-08

Table 13: SLACHCIH(7) Time Growth

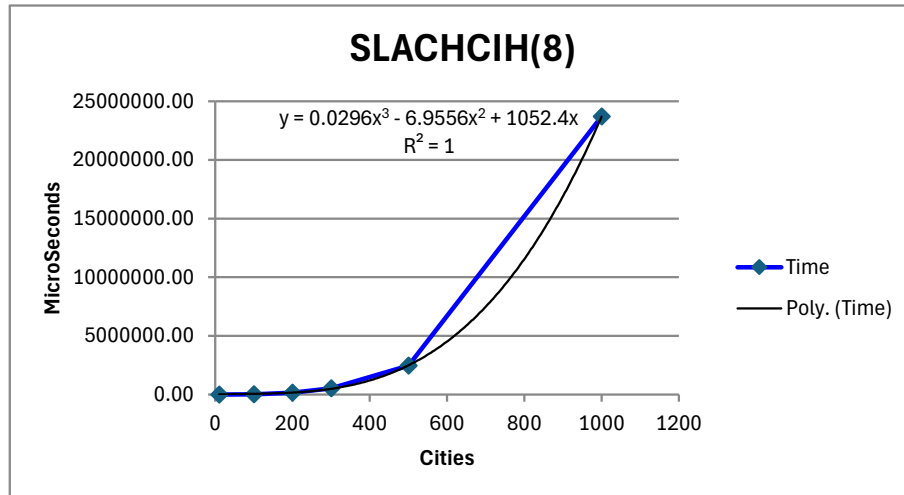


Figure 54: SLACHCIH(8) Trend Line

The value the look-ahead depth has is the difference between the algorithm time complexity being $O(n^3)$ (when the depth is zero), and $O(n^5)$ (when depth is n); this is the same as in Equation4. The CIH part of the algorithm is a constant $O(n^3)$ time complexity, while the look-ahead cost is based on the depth. This is observed in the data by changing the depths (Figure55); the difference in time complexity is $O(n^2)$, which matches the cost of the look-ahead function.

The jump from depth zero to depth one is abrupt; this abnormally large increase is theorised as the cost of invoking the *ShortestRoute()* function, which would only be fully invoked if depth is greater than 0. This is noticeable in Figure56 between the violet and indigo lines.

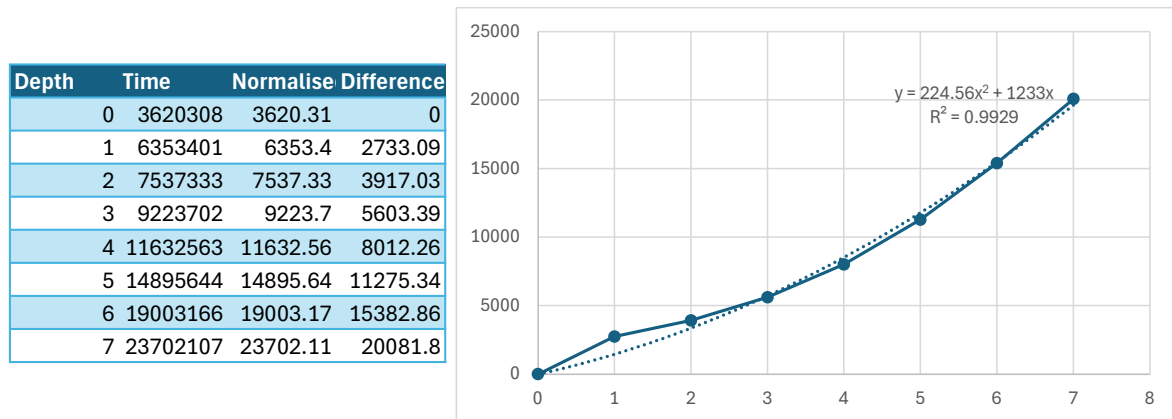


Figure 55: Depth Trend Line

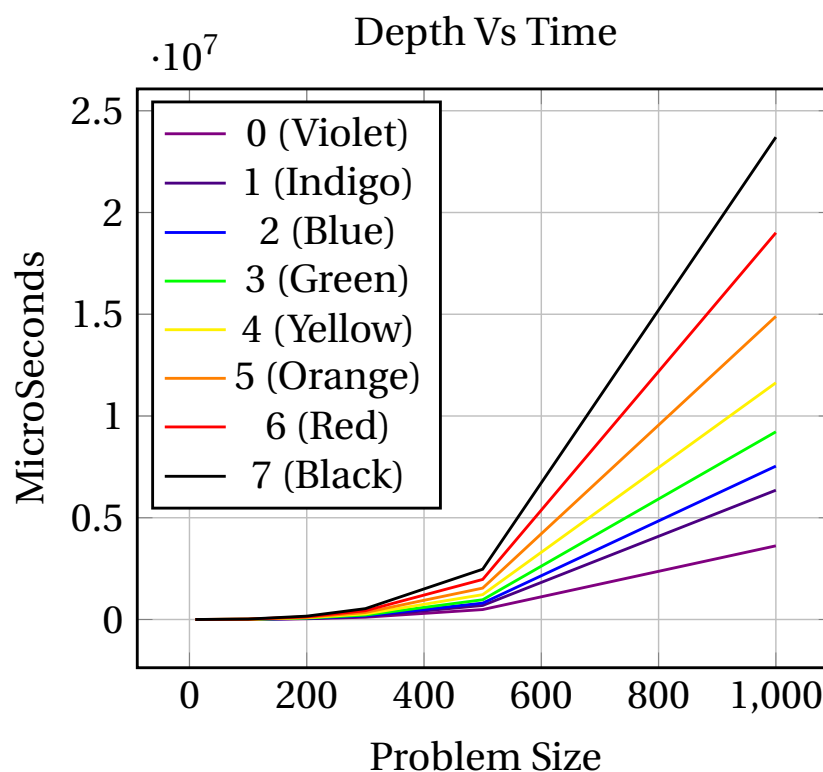


Figure 56: Plot Of Depth Vs Time

12 Conclusions

Overall, this project has demonstrated that the Look-Ahead Cheapest Insertion Heuristic is a worthy heuristic in solving the Travelling Salesman Problem.

The project findings demonstrated consistent and significant gains in the solution result by applying a look-ahead design to the Cheapest Insertion Heuristic to avoid local pitfalls. A new algorithm, Dynamic Look-Ahead Convex Hull Cheapest Insertion Heuristic(DLACHCIH), produced a better result than conventional heuristics on average:

Algorithm	Performance Difference
Convex Hull Cheapest Insertion Heuristic	1.1482%
Cheapest Insertion Heuristic	4.8412%
Nearest Neighbour Search	3.4912%

Note: higher % is better

Table 14: Performance Of DLACHCIH vs Conventional Heuristics

In the research of TSP, any gains are a great accomplishment to theorists, and the difference is better than expected. It proved the project motivation that a look-ahead design can be used to get a better solution result at the cost of computational performance.

DLACHCIH was only 7.2345% off of the exact solutions to TSPLIB(Table12), which proves it could be used as valuable inputs as the starting solution to the Concorde TSP Solver(Section3.1.1). Practitioners particularly care about optimal solutions for the time-saving benefits gained by providing a better lower bound for the TSP Concorde Solver. The time complexity has been thoroughly evaluated at Big O of $O(n^5)$, but a more practical complexity of n^3 is realistic(Figure56), which is in line with other heuristics. The algorithms presented in this report are suitable replacements for CIH and classical heuristic; however, they should not be used in real-world use cases(explained below).

Software engineering principles and advanced designs(Section6) have been applied to all areas of the codebase to solve the project's needs. Essential challenges in C++(Section7) and features(Section8) have been solved to ensure the validity of the work. Careful consideration into its development with the use of waterfall project management(Section9) and extensive value boundary unit testing(Section10) ensured the project's success. The software is robust, interoperable, and highly efficient, is valuable in benchmarking research. Multiple aims(Table2) have been asked for to provide validity and functionality to the project, which have been solved with extensive continuous development over eight months of work. The core C++

template library is optimised, functional, and easily supports various configurations. It could be configured in 96 unique setups, but only seven were evaluated because of a lack of computational resources.

Most important was the in-depth research(Section3) into the Travelling Salesman Problem and algorithmic design. Using the proper methodology(Section5) to approach a meaningful and unbiased evaluation(Section11) was crucial. The evaluation draws many valuable conclusions about the time complexity and solution quality. Even after solving 114,000 TSP problems, it was not as in-depth as it could have been due to the lengthy run times on the available hardware that the project had to supply itself.

Referring back to the aims(Section1.2) laid out for this report, each goal has been achieved:

1. SLACIH and DLACIH have been designed, implemented, and tested. Both run in polynomial time and are based upon the Cheapest Insertion Heuristic.
2. A testing framework has been built that records distance & time fairly for every TSP algorithm. Empirical testing and ubiquitous TSPLIB are used for objective data collection against other heuristic and exact solutions.
3. A sizeable and intricate system for fulfilling many needs with interoperability in mind. Multiple algorithms can be run with no crashes or faults.

The algorithms presented in this report should not be used for real-world cases; it was purely researched to explore the possibility of improving greedy algorithms. It was known before the project that no technological advancements would be made due to the fierce research focus on the Travelling Salesman Problem. The current leading benchmarking heuristic, Lin-Kernighan Heuristic(LKH)^[81], supersedes this research. Their heuristic runs in $O(n^3)$ and achieves a solution within 0.5% of an exact solution^[129]. However, the aim of this project was to improve upon classical heuristics and prove that a look-ahead design is beneficial to the Cheapest Insertion Heuristic, which has been the case.

12.1 Future Work

With unlimited resources, I would be able to do the following addition of multi-threaded and high-performance computing features to the algorithm. The algorithm's iterative nature allows for parallelisation at multiple stages, and with libraries like OpenMP^[130], development would be quick and easy. The difficulty lies in having the hardware and time required to gather the large amount of data needed.

Although DLACHCIH matched the best result of SLACHCIH(Table9), this was only done with the simplest design for doing dynamic look-ahead. Gradient descent was used to optimise DLACHCIH but was far from optimally implemented and had room for improvement. Other AI techniques(two-stage neural networks) could be used to evaluate the best depth and explore the different designs proposed in Section4.4, specifically the adaptive dynamic look-ahead, which would almost certainly see better results.

A significant reason for the research into heuristics is the use as a starting solution for the Concorde TSP Solver. Attaching the Concorde API to the code base and measuring the timing difference between LKH would provide more meaningful research to practitioners, but would require building a system to the University of Waterloo's specifications.

The algorithm design is challenging to follow intuitively, and the static GUI outputs and low graphic fidelity could be improved to help illustrate the design. Applying animation would be a worthwhile addition to the project, but would present a difficult challenge.

12.2 Closing Remark

I am proud of the work I produced; it has expanded my knowledge, skills, and work ethics. I want to thank my supervisor, Dr Rajish Chitnis, for his constant support and guidance in the project. Also, I want to thank my family and friends for their support.

References

- [1] M. M. Flood, “The traveling-salesman problem,” *Operations Research*, vol. 4, no. 1, pp. 61–75, 1956. [Online]. Available: <http://www.jstor.org/stable/167517>
- [2] A. Mazidi and E. Damghanijazi, “Meta-heuristic approaches for solving travelling salesman problem,” *International Journal of Advanced Research in Computer Science*, vol. 5, 03 2017.
- [3] S. H. Rubin, T. Bouabana-Tebibel, Y. Hoadjli, and Z. Ghalem, “Reusing the np-hard traveling-salesman problem to demonstrate that p np (invited paper),” in *2016 IEEE 17th International Conference on Information Reuse and Integration (IRI)*, 2016, pp. 574–581.
- [4] G. Reinelt, “TSPLIB—a traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [5] BCS, The Chartered Institute for IT, “BCS Code of Conduct,” 2024. [Online]. Available: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>
- [6] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, ser. STOC '71. New York, NY, USA: Association for Computing Machinery, 1971, p. 151–158. [Online]. Available: <https://doi.org/10.1145/800157.805047>
- [7] G. Dantzig, R. Fulkerson, and S. Johnson, “Solution of a large-scale traveling-salesman problem,” *Journal of the Operations Research Society of America*, vol. 2, no. 4, pp. 393–410, 1954. [Online]. Available: <http://www.jstor.org/stable/166695>
- [8] N. DQ, “Vertex definition,” *Maths Insight*, 2024. [Online]. Available: http://mathinsight.org/definition/graph_vertex
- [9] N. Biggs, E. Lloyd, and R. Wilson, *Graph Theory, 1736-1936*. Clarendon Press, 1986. [Online]. Available: <https://books.google.co.uk/books?id=XqYTk0sXmpoC>
- [10] G. A. Croes, “A method for solving traveling-salesman problems,” *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958. [Online]. Available: <http://www.jstor.org/stable/167074>

- [11] M. Held and R. M. Karp, "The traveling-salesman problem and minimum spanning trees," *Operations Research*, vol. 18, no. 6, pp. 1138–1162, 1970. [Online]. Available: <http://www.jstor.org/stable/169411>
- [12] Wikipidia, "Hamiltonian path," *Wikipidia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Hamiltonian_path
- [13] O. Gutiérrez, E. Zamora, and R. Menchaca, "Graph representation for learning the traveling salesman problem," in *Pattern Recognition*, E. Roman-Rangel, Á. F. Kuri-Morales, J. F. Martínez-Trinidad, J. A. Carrasco-Ochoa, and J. A. Olvera-López, Eds. Cham: Springer International Publishing, 2021, pp. 153–162.
- [14] K. Ilavarasi and K. S. Joseph, "Variants of travelling salesman problem: A survey," in *International Conference on Information Communication and Embedded Systems (ICICES2014)*, 2014, pp. 1–7.
- [15] P. Vána and J. Faigl, "On the dubins traveling salesman problem with neighborhoods," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 4029–4034.
- [16] H. Jiang, J. Li, D. Liu, and S. Singla, "Algorithms and adaptivity gaps for stochastic k -tsp," 2019. [Online]. Available: <https://arxiv.org/abs/1911.02506>
- [17] M. López-Ibáñez, C. Blum, J. W. Ohlmann, and B. W. Thomas, "The travelling salesman problem with time windows: Adapting algorithms from travel-time to makespan optimization," *Applied Soft Computing*, vol. 13, no. 9, pp. 3806–3815, 2013. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494613001658>
- [18] D. G. Macharet, "Dubins tsp picture," 2025. [Online]. Available: <https://homepages.dcc.ufmg.br/~doug/site/en/research/>
- [19] K. Helsgaun, "k-tsp picture," 2025. [Online]. Available: <http://webhotel4.ruc.dk/~keld/research/LKH-3/>
- [20] S. Carmesin, D. Woller, D. Parker, M. Kulich, and M. Mansouri, "The hamiltonian cycle and travelling salesperson problems with traversal-dependent edge deletion," *Journal of Computational Science*, vol. 74, p. 102156, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877750323002168>

- [21] J. Le Ny, E. Feron, and E. Frazzoli, "On the dubins traveling salesman problem," *IEEE Trans. Automat. Contr.*, vol. 57, pp. 265–270, 01 2012.
- [22] C. Nilsson, "Heuristics for the traveling salesman problem."
- [23] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, *An analysis of several heuristics for the traveling salesman problem*. Dordrecht: Springer Netherlands, 2009, pp. 45–69. [Online]. Available: https://doi.org/10.1007/978-1-4020-9688-4_3
- [24] R. Bixby and W. Cook, "Finding tours in the tsp," 05 2000.
- [25] M. Englert, H. Röglin, and B. Vöcking, "Worst case and probabilistic analysis of the 2-opt algorithm for the tsp," *Algorithmica*, vol. 68, no. 1, pp. 190–264, Jan 2014. [Online]. Available: <https://doi.org/10.1007/s00453-013-9801-4>
- [26] B. Jeromin and F. Körner, "Triangle inequality and symmetry in connection with the assignment and the traveling salesman problem," *European Journal of Operational Research*, vol. 38, no. 1, pp. 70–75, 1989. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0377221789904700>
- [27] K. Kalmanson, "Edgeconvex circuits and the traveling salesman problem," *Canadian Journal of Mathematics*, vol. 27, no. 5, p. 1000–1010, 1975.
- [28] Brilliant, "triangle inequality picture," 2025. [Online]. Available: <https://brilliant.org/wiki/triangle-inequality/>
- [29] R. Capelli, S. Caracciolo, A. Di Gioacchino, and E. M. Malatesta, "Exact value for the average optimal cost of the bipartite traveling salesman and two-factor problems in two dimensions," *Physical Review E*, vol. 98, no. 3, Sep. 2018. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevE.98.030101>
- [30] J. Y. Lee, S.-Y. Shin, T. H. Park, and B.-T. Zhang, "Solving traveling salesman problems with dna molecules encoding numerical values," *Biosystems*, vol. 78, no. 1, pp. 39–47, 2004. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0303264704001157>
- [31] G. J. Woeginger, *Exact Algorithms for NP-Hard Problems: A Survey*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 185–207. [Online]. Available: https://doi.org/10.1007/3-540-36478-1_17

- [32] A. Neoh, H. Chen, and C. Chase, "An evaluation of the traveling salesman problem," 2020.
- [33] C. Jewell, C. Parlato, J. Gibson, and Y. Fazea, "Time optimization algorithm for traveling salesman problem in communication networks," in *2024 IEEE Opportunity Research Scholars Symposium (ORSS)*, 2024, pp. 92–96.
- [34] M. S. Bakshi and S. R. Arora, "The sequencing problem," *Management Science*, vol. 16, no. 4, pp. B247–B263, 1969. [Online]. Available: <http://www.jstor.org/stable/2628801>
- [35] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962. [Online]. Available: <http://www.jstor.org/stable/2098806>
- [36] R. Bellman, "Dynamic programming treatment of the travelling salesman problem," *J. ACM*, vol. 9, no. 1, p. 61–63, Jan. 1962. [Online]. Available: <https://doi.org/10.1145/321105.321111>
- [37] S. Helwig, F. Hüffner, I. Rössling, and M. Weinard, "Chapter 3. selected design issues," 10 2010.
- [38] H. W. Lenstra, "Integer programming with a fixed number of variables," *Mathematics of Operations Research*, vol. 8, no. 4, pp. 538–548, 1983. [Online]. Available: <http://www.jstor.org/stable/3689168>
- [39] M. Diaby, "The traveling salesman problem: A linear programming formulation," *arXiv preprint cs/0609005*, 2006. [Online]. Available: <https://arxiv.org/abs/cs/0609005>
- [40] H. H. Hoos and T. Stützle, "On the empirical scaling of run-time for finding optimal solutions to the travelling salesman problem," *European Journal of Operational Research*, vol. 238, no. 1, pp. 87–94, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221714002811>
- [41] D. Applegate, R. Bixby, V. Chvatal, and W. Cook, "Concorde tsp solver," <https://www.math.uwaterloo.ca/tsp/concorde.html>, 2025, accessed: 2025-03-26.
- [42] D. L. Applegate, R. E. Bixby, V. Chvátal, W. Cook, D. G. Espinoza, M. Goycoolea, and K. Helsgaun, "Certification of an optimal tsp tour through 85,900 cities," *Operations Research Letters*, vol. 37, no. 1, pp. 11–15, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167637708001132>

- [43] D. Applegate, R. Bixby, V. Chvátal, and W. Cook, “Concorde tsp solver benchmarks,” <https://www.math.uwaterloo.ca/tsp/concorde/benchmarks/bench99.html>, 2025, accessed: 2025-03-26.
- [44] —, “Implementing the dantzig-fulkerson-johnson algorithm for large traveling salesman problems,” *Mathematical Programming*, vol. 97, no. 1-2, pp. 33–48, 2003. [Online]. Available: <https://link.springer.com/article/10.1007/s10107-003-0440-4>
- [45] —, “Amazon delivery tsp challenge,” <https://www.math.uwaterloo.ca/tsp/amz/index.html>, 2025, accessed: 2025-03-26.
- [46] D. L. Applegate, W. Cook, S. Dash, and D. G. Espinoza, “Exact solutions to linear programming problems,” *Operations Research Letters*, vol. 35, no. 6, pp. 693–699, 2007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167637707000211>
- [47] D. Applegate, R. Bixby, V. Chvátal, and W. Cook, “On the solution of traveling salesman problems,” *Documenta Mathematica, Extra Volume ICM*, pp. 645–656, 1998. [Online]. Available: https://www.math.uwaterloo.ca/~bico/papers/tsp_icm.pdf
- [48] —, *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ: Princeton University Press, 2006.
- [49] R. Guang and S. Khodadadian, “Linear function approximation as a resource efficient method to solve the travelling salesman problem,” *Journal of Student Research*, vol. 10, 06 2022.
- [50] W. J. Cook, *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ: Princeton University Press, 2012.
- [51] D. R. Morrison, S. H. Jacobson, J. J. Sauppe, and E. C. Sewell, “Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning,” *Discrete Optimization*, vol. 19, pp. 79–102, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1572528616000062>
- [52] T. Volgenant and R. Jonker, “A branch and bound algorithm for the symmetric traveling salesman problem based on the 1-tree relaxation,” *European Journal of Operational Research*, vol. 9, no. 1, pp. 83–89, 1982. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0377221782900157>

- [53] C. University, “Branch and bound (bb) for minlp,” https://optimization.cbe.cornell.edu/index.php?title=Branch_and_bound_%28BB%29_for_MINLP, 2025, accessed: 2025-03-26.
- [54] J. E. Kelley, Jr., “The cutting-plane method for solving convex programs,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 4, pp. 703–712, 1960. [Online]. Available: <https://doi.org/10.1137/0108053>
- [55] B. Fleischmann, “A cutting plane procedure for the travelling salesman problem on road networks,” *European Journal of Operational Research*, vol. 21, no. 3, pp. 307–317, 1985. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0377221785901511>
- [56] W. contributors, “Cutting-plane method,” https://en.wikipedia.org/wiki/Cutting-plane_method, 2025, accessed: 2025-03-26.
- [57] J. E. Mitchell, “Branch-and-cut algorithms for combinatorial optimization problems,” *Handbook of applied optimization*, vol. 1, no. 1, pp. 65–77, 2002.
- [58] M. Padberg and G. Rinaldi, “A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems,” *SIAM Review*, vol. 33, no. 1, pp. 60–100, 1991. [Online]. Available: <http://www.jstor.org/stable/2030652>
- [59] Y. Lu, J.-K. Hao, and Q. Wu, “Solving the clustered traveling salesman problem via tsp methods,” 2022. [Online]. Available: <https://arxiv.org/abs/2007.05254>
- [60] J. Zhang, C. Liu, X. Li, H.-L. Zhen, M. Yuan, Y. Li, and J. Yan, “A survey for solving mixed integer programming via machine learning,” *Neurocomputing*, vol. 519, pp. 205–217, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231222014035>
- [61] J. D. C. Little, K. G. Murty, D. W. Sweeney, and C. Karel, “An algorithm for the traveling salesman problem,” *Operations Research*, vol. 11, no. 6, pp. 972–989, 1963. [Online]. Available: <http://www.jstor.org/stable/167836>
- [62] S. Desale, A. Rasool, S. Andhale, and P. Rane, “Heuristic and meta-heuristic algorithms and their relevance to the real world: a survey,” *Int. J. Comput. Eng. Res. Trends*, vol. 351, no. 5, pp. 2349–7084, 2015.

- [63] N. Kokash, "An introduction to heuristic algorithms," *Department of Informatics and Telecommunications*, vol. 1, pp. 1–7, 2005.
- [64] D. Jungnickel, *The Greedy Algorithm*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 135–161. [Online]. Available: https://doi.org/10.1007/978-3-642-32278-5_5
- [65] J. Bang-Jensen, G. Gutin, and A. Yeo, "When the greedy algorithm fails," *Discrete Optimization*, vol. 1, no. 2, pp. 121–127, 2004. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1572528604000222>
- [66] V. V. Vazirani, *Approximation Algorithms*. Berlin, Heidelberg: Springer, 2001. [Online]. Available: <https://link.springer.com/book/10.1007/3-540-36478-1>
- [67] G. Kizilates and F. Nuriyeva, "On the nearest neighbor algorithms for the traveling salesman problem," in *Advances in Computational Science, Engineering and Information Technology*, D. Nagamalai, A. Kumar, and A. Annamalai, Eds. Heidelberg: Springer International Publishing, 2013, pp. 111–118.
- [68] J. Jones and A. Adamatzky, "Computation of the travelling salesman problem by a shrinking blob," *Natural Computing*, vol. 13, no. 1, pp. 1–16, Mar 2014. [Online]. Available: <https://doi.org/10.1007/s11047-013-9401-x>
- [69] —, "Computation of the travelling salesman problem by a shrinking blob," 2013. [Online]. Available: <https://arxiv.org/abs/1303.4969>
- [70] F. Fargiana, R. Respitawulan, Y. Fajar, D. Suhaedi, and E. Harahap, "Implementation of cheapest insertion heuristic algorithm in determining shortest delivery route," *International Journal of Global Operations Research*, vol. 3, pp. 37–45, 05 2022.
- [71] S. Meeran and A. Share, "Optimum path planning using convex hull and local search heuristic algorithms," *Mechatronics*, vol. 7, no. 8, pp. 737–756, 1997. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957415897000330>
- [72] S. R. Lay, *Convex Sets and Their Applications*. New York: John Wiley & Sons, 1982.
- [73] R. Liu, Y. Tang, and P. Chan, "A fast convex hull algorithm inspired by human visual perception," *Multimedia Tools and Applications*, vol. 77, 12 2018.
- [74] K. H. Borgwardt, "Average complexity of a gift-wrapping algorithm for determining the convex hull of randomly given points," *Discrete & Computational Geometry*, vol. 17, pp. 79–109, 1997.

- [75] R. Graham, "An efficient algorithm for determining the convex hull of a finite planar set," *Information Processing Letters*, vol. 1, no. 4, pp. 132–133, 1972. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0020019072900452>
- [76] T. M. Chan, "Optimal output-sensitive convex hull algorithms in two and three dimensions," *Discrete & Computational Geometry*, vol. 16, no. 4, pp. 361–368, Apr 1996. [Online]. Available: <https://doi.org/10.1007/BF02712873>
- [77] S. Murthy and S. Salzberg, "Lookahead and pathology in decision tree induction," in *IJCAI*. Citeseer, 1995, pp. 1025–1033.
- [78] D. Frost, R. Dechter *et al.*, "Look-ahead value ordering for constraint satisfaction problems," in *IJCAI (1)*. Citeseer, 1995, pp. 572–578.
- [79] M. Dong and R. Kothari, "Look-ahead based fuzzy decision tree induction," *IEEE Transactions on Fuzzy Systems*, vol. 9, no. 3, pp. 461–468, 2001.
- [80] U. Sarkar, P. Chakrabarti, S. Ghose, and S. Desarkar, "Improving greedy algorithms by lookahead-search," *Journal of Algorithms*, vol. 16, no. 1, pp. 1–23, 1994. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0196677484710017>
- [81] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling-salesman problem," *Operations Research*, vol. 21, no. 2, pp. 498–516, 1973. [Online]. Available: <http://www.jstor.org/stable/169020>
- [82] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.
- [83] I. Cohen, C. Epstein, P. Isaiah, S. Kuzi, and T. Shima, "Discretization-based and look-ahead algorithms for the dubins traveling salesperson problem," *IEEE Transactions on Automation Science and Engineering*, vol. 14, no. 1, pp. 383–390, 2017.
- [84] D. Karhi and D. E. Tamir, "Caching in the tsp search space," in *Next-Generation Applied Intelligence*, B.-C. Chien, T.-P. Hong, S.-M. Chen, and M. Ali, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 221–230.
- [85] R. M. Karp, "Probabilistic analysis of partitioning algorithms for the traveling-salesman problem in the plane," *Mathematics of operations research*, vol. 2, no. 3, pp. 209–224, 1977.

- [86] J.-Q. Yang, J.-G. Yang, and G.-L. Chen, "Solving large-scale tsp using adaptive clustering method," in *2009 Second International Symposium on Computational Intelligence and Design*, vol. 1, 2009, pp. 49–51.
- [87] S. Arora, "Polynomial time approximation schemes for euclidean tsp and other geometric problems," in *Proceedings of 37th Conference on Foundations of Computer Science*, 1996, pp. 2–11.
- [88] F. Lin and H.-P. Hsieh, "A grid-based two-stage parallel matching framework for bi-objective euclidean traveling salesman problem," *ACM Transactions on Spatial Algorithms and Systems*, vol. 8, no. 4, pp. 1–33, 2022.
- [89] A. A. Khan, M. U. Khan, and M. Iqbal, "Multilevel graph partitioning scheme to solve traveling salesman problem," in *2012 Ninth International Conference on Information Technology - New Generations*, 2012, pp. 458–463.
- [90] A. Jaradat, B. Matalkeh, and W. Diabat, "Solving traveling salesman problem using firefly algorithm and k-means clustering," in *2019 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*, 2019, pp. 586–589.
- [91] S. Arora, "Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems," *Journal of the ACM (JACM)*, vol. 45, no. 5, pp. 753–782, 1998.
- [92] K. Krishna and M. Narasimha Murty, "Genetic k-means algorithm," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 29, no. 3, pp. 433–439, 1999.
- [93] S.-L. Developers, "K-means clustering on digits dataset example," https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_digits.html, accessed: 2025-03-27.
- [94] H. Samet, "The quadtree and related hierarchical data structures," *ACM Computing Surveys (CSUR)*, vol. 16, no. 2, pp. 187–260, 1984.
- [95] M. J. Quinn, *Parallel computing (2nd ed.): theory and practice*. USA: McGraw-Hill, Inc., 1994.
- [96] C. A. Navarro, N. Hitschfeld-Kahler, and L. Mateu, "A survey on parallel computing and its applications in data-parallel problems using gpu architectures," *Communications in Computational Physics*, vol. 15, no. 2, p. 285–329, 2014.

- [97] G. A. Kindervater, J. K. Lenstra, and D. B. Shmoys, "The parallel complexity of tsp heuristics," *Journal of Algorithms*, vol. 10, no. 2, pp. 249–270, 1989. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0196677489900151>
- [98] N. Fujimoto and S. Tsutsui, "A highly-parallel tsp solver for a gpu computing platform," in *Numerical Methods and Applications*, I. Dimov, S. Dimova, and N. Kolkovska, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 264–271.
- [99] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/000437028090051X>
- [100] B. Sharma, R. Mohanty, and S. Panda, "A note on lookahead in real life and computing," 2024. [Online]. Available: <https://arxiv.org/abs/2403.17942>
- [101] G. Gutin, A. Yeo, and A. Zverovich, "Traveling salesman should not be greedy: domination analysis of greedy-type heuristics for the tsp," *Discrete Applied Mathematics*, vol. 117, no. 1, pp. 81–86, 2002. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0166218X01001950>
- [102] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat, and R. Schwartz, "Dynamic speculation lookahead accelerates speculative decoding of large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2405.04304>
- [103] G. Strimel, Y. Xie, B. King, M. Radfar, A. Rastrow, and T. Mouchtaris, "Lookahead when it matters: Adaptive non-causal transformers for streaming neural transducers," 2023. [Online]. Available: <https://www.amazon.science/publications/lookahead-when-it-matters-adaptive-non-causal-transformers-for-streaming-neural-transducers>
- [104] Z. Yahya. Assoc Prof Ts. Dr, R. Wirza, K. Fatimah, A. Rizaan, and A. Rizal, "A concave hull based algorithm for object shape reconstruction," *International Journal of Information Technology and Computer Science*, vol. 09, pp. 1–3, 01 2015.
- [105] A. H. Halim, I. Ismail, and S. Das, "Performance assessment of the metaheuristic optimization algorithms: an exhaustive review," *Artificial Intelligence Review*, vol. 54, no. 3, pp. 2323–2409, Mar 2021. [Online]. Available: <https://doi.org/10.1007/s10462-020-09906-6>
- [106] R. L. Rardin and R. Uzsoy, "Experimental evaluation of heuristic optimization algorithms: A tutorial," *Journal of Heuristics*, vol. 7, pp. 261–304, 2001.

- [107] M. L. Patten, *Proposing empirical research: A guide to the fundamentals*. Routledge, 2016.
- [108] K. Bhattacharjee and S. Das, “A search for good pseudo-random number generators: Survey and empirical studies,” *Computer Science Review*, vol. 45, p. 100471, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013722000144>
- [109] G. Reinelt, “TSPLIB: A library of sample instances for the TSP (and related problems),” 1995, accessed: 2025-04-02. [Online]. Available: <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>
- [110] D. Tosoni, C. Galli, T. Hanne, and R. Dornberger, “Benchmarking metaheuristic optimization algorithms on travelling salesman problems,” in *Proceedings of the 8th International Conference on E-Society, e-Learning and e-Technologies*, ser. ICSLT '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 20–25. [Online]. Available: <https://doi.org/10.1145/3545922.3545926>
- [111] V. Chvátal, “Traveling salesman problem: Methods and applications,” Online, 2001. [Online]. Available: <https://www.math.uwaterloo.ca/tsp/methods/talks/chvatal01/slide2.html>
- [112] L. Bormane, J. Gržibovska, S. Bērziša, and J. Grabis, “Impact of requirements elicitation processes on success of information system development projects,” *Information Technology and Management Science*, vol. 19, 12 2016.
- [113] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [114] Microsoft, “C++ development with visual studio,” <https://visualstudio.microsoft.com/vs/features/cplusplus/>, 2025.
- [115] cppreference.com, “C++20 - cppreference.com,” 2024, accessed: 2025-04-06. [Online]. Available: <https://en.cppreference.com/w/cpp/20>
- [116] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Tech. Rep. 1, 2007, accessed: 2025-04-04. [Online]. Available: <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- [117] “A study of real-time memory management: Evaluating operating system’s performance,” *Automatyka/Automatics*, vol. 17, no. 1, p. 29, 2015. [Online]. Available: <https://doi.org/10.7494/automat.2013.17.1.29>

- [118] A. Shvets. (n.d.) Builder design pattern. Refactoring.Guru. [Online]. Available: <https://refactoring.guru/design-patterns/builder>
- [119] D. Vandevoorde and N. M. Josuttis, *C++ templates: the complete guide*. Addison-Wesley Professional, 2002.
- [120] D. Abrahams and A. Gurtovoy, *C++ template metaprogramming: concepts, tools, and techniques from Boost and beyond*. Pearson Education, 2004.
- [121] The Qt Company, “Qt 6 - product overview,” <https://www.qt.io/product/qt6>, 2024.
- [122] J. McNamara, “libxlsxwriter - a c library for creating excel xlsx files,” <https://github.com/jmcnamara/libxlsxwriter>, 2014.
- [123] M. Galassi, J. Davies, J. Theiler, B. Gough, G. Jungman, M. Booth, and F. Rossi, *GNU Scientific Library Reference Manual*, 3rd ed., GNU Project, 2021, version 2.7. [Online]. Available: <https://www.gnu.org/software/gsl/>
- [124] GoogleTest, “Googletest - google testing and mocking framework,” <https://github.com/google/googletest>, 2025.
- [125] R. Osherove, *The Art of Unit Testing: with examples in C*. Simon and Schuster, 2013.
- [126] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” <https://www.rfc-editor.org/info/rfc8259>, 2017, rFC 8259.
- [127] M. H. Ibrahim, “Reflection in object-oriented programming,” *International Journal on Artificial Intelligence Tools*, vol. 01, no. 01, pp. 117–136, 1992. [Online]. Available: <https://doi.org/10.1142/S0218213092000156>
- [128] T. Thesing, C. Feldmann, and M. Burchardt, “Agile versus waterfall project management: decision model for selecting the appropriate approach to a project,” *Procedia Computer Science*, vol. 181, pp. 746–756, 2021.
- [129] K. Helsgaun, “An effective implementation of the lin–kernighan traveling salesman heuristic,” *European Journal of Operational Research*, vol. 126, no. 1, pp. 106–130, 2000. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221799002842>
- [130] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Version 5.2*, OpenMP Architecture Review Board, 2023, <https://www.openmp.org/specifications/>.